

CS336 作业 1 (基础): 构建 Transformer 语言模型

版本 26.0.3
CS336 课程团队
2026 年春季

1 作业概览

本次作业中, 你将从零开始构建训练标准 Transformer 语言模型 (LM) 所需的全部组件, 并训练一些模型。

你将实现的内容

1. 字节对编码 (BPE) 分词器 (第 2 节)。
2. Transformer 语言模型 (LM) (第 3 节)。
3. 交叉熵损失函数和 AdamW 优化器 (第 4 节)。
4. 训练循环, 支持模型状态与优化器状态的序列化和加载 (第 5 节)。

你将运行的内容

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 使用训练好的分词器对数据集进行编码, 将其转换为整数 ID 序列。
3. 在 TinyStories 数据集上训练 Transformer 语言模型。
4. 使用训练好的 Transformer 语言模型生成样本, 并评估困惑度。
5. 在 OpenWebText 上训练模型, 并将获得的困惑度提交到排行榜。

你可以使用的内容

我们要求你从零开始构建每个组件。具体而言, 除以下内容外, 你不能使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义:

- `torch.nn.Parameter`
- `torch.nn` 中的容器类 (如 `Module`、`ModuleList`、`Sequential` 等)。¹
- `torch.optim.Optimizer` 基类

你可以使用 PyTorch 中的其他任何定义。如果你想使用某个函数或类, 但不确定是否允许, 请随时在 Slack 上提问。拿不准时, 请考虑使用它是否违背了本作业“从零实现”的精神。

关于 AI 工具的声明

AI 能够完全自主地解决作业中的许多部分。这会让你更难深入投入课程材料, 并从中学习。

允许使用 AI 工具回答高层次的概念问题, 或提供函数签名、库 API 等底层编程文档信息。但是, **不允许使用 AI 工具实现任何作业的任何部分**。这既包括编程智能体 (例如 Cursor Agents、Codex、Claude Code), 也包括 AI 自动补全 (例如 Cursor Tab、GitHub Copilot)。使用 AI 智能体时, 请确保其使用所提供的 AGENTS.md 文件。使用聊天机器人时, 也应包含该提示词。

¹ 完整列表见 <https://pytorch.org/docs/stable/nn.html#containers>。

我们强烈建议你在完成作业时关闭 IDE 中的 AI 自动补全（例如 Cursor Tab、GitHub Copilot）；不过，非 AI 自动补全完全可以使用，例如补全函数名称。以往的同学特别提到，关闭 AI 自动补全让他们更容易深入投入课程材料。

完整的 AI 使用政策见这份文档。

代码的组织方式

作业代码以及本作业说明均托管在 GitHub 上：

<https://github.com/stanford-cs336/assignment1-basics>

请使用 `git clone` 克隆该仓库。如有更新，我们会通知你，届时可运行 `git pull` 获取最新内容。

1. `cs336_basics/*`：你在这里编写代码。注意，这个目录中没有代码——你可以完全从零开始，自由组织实现！
2. `adapters.py`：你的代码必须提供一组指定功能。对于每项功能（例如缩放点积注意力），只需调用你自己的代码，填入相应实现（例如 `run_scaled_dot_product_attention`）。注意：你对 `adapters.py` 的修改不应包含任何实质性逻辑；这里只应包含衔接代码。
3. `test_*.py`：包含所有你必须通过的测试（例如 `test_scaled_dot_product_attention`）。这些测试会调用 `adapters.py` 中定义的接口。不要修改测试文件。

如何提交

提交时，运行 `make_submission.sh` 生成提交用的 `zip` 压缩包。如果有大型数据文件或检查点不希望包含在提交压缩包中，请务必将相应文件加入该脚本的排除列表。

你需要向 Gradescope 提交以下文件：

- `writeup.pdf`：回答所有书面问题。请对答案进行排版。
- `code.zip`：包含你编写的全部代码。

要提交到排行榜，请向下列仓库提交 PR：

<https://github.com/stanford-cs336/assignment1-basics-leaderboard>

详细提交说明见排行榜仓库中的 `README.md`。

从哪里获取数据集

本作业使用两个经过预处理的数据集：TinyStories [R. Eldan 等, 2023] 和 OpenWebText [A. Gokaslan 等, 2019]。两个数据集都各自存储为一个大型纯文本文件。

如果你是本课程的在校学生，可在计算资源指南中找到数据集下载说明。

如果你在家自学，可使用 `README.md` 中的命令下载这些文件。

低资源提示：开始之前

在本课程的作业说明中，我们会提供建议，帮助你在 GPU 资源较少或没有 GPU 的情况下完成部分作业。例如，我们有时会建议缩小数据集或模型规模，或者解释如何在 Mac 的

集成 GPU 或 CPU 上运行训练代码。这些“低资源提示”都放在蓝色边框中（就像这个框）。即使你是已选课、能够使用课程机器的斯坦福学生，这些提示也可能帮助你更快迭代、节省时间，因此我们建议你阅读！

低资源提示：在 Apple Silicon 或 CPU 上完成作业 1

使用课程团队的参考实现，在配备 36 GB 内存的 Apple M4 Max 芯片上，我们可以训练一个能够生成相当流畅文本的语言模型：使用 Metal GPU (MPS) 耗时不到 5 分钟，使用 CPU 则约需 30 分钟。即使你不太理解这些术语，也不用担心！只要知道，如果你有一台比较新的笔记本电脑，而且实现正确、高效，就能够训练一个小型语言模型，以不错的流畅程度生成简单的儿童故事。

在作业后面的部分，我们会说明使用 CPU 或 MPS 时需要做哪些修改。

2 字节对编码 (BPE) 分词器

在作业的第一部分，我们将训练并实现一个字节级字节对编码 (BPE) 分词器 [R. Sennrich 等, 2016; C. Wang 等, 2019]。具体而言，我们会将任意 (Unicode) 字符串表示为字节序列，并在该字节序列上训练 BPE 分词器。之后，我们将使用该分词器把文本 (字符串) 编码为词元 (token，即整数序列)，用于语言建模。

2.1 Unicode 标准

Unicode 是一种文本编码标准，将字符映射为整数码点。截至 Unicode 17.0 (2025 年 9 月发布)，该标准在 172 种文字系统中定义了 159,801 个字符。例如，字符“s”的码点是 115 (通常记作 U+0073，其中 U+ 是约定俗成的前缀，0073 是 115 的十六进制表示)，字符“牛”的码点是 29275。在 Python 中，可以使用 `ord()` 函数将单个 Unicode 字符转换为对应整数。`chr()` 函数则将整数 Unicode 码点转换为包含相应字符的字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

问题 (unicode1)：理解 Unicode (1 分)

(a) `chr(0)` 返回哪个 Unicode 字符？

提交内容：一句话回答。

(b) 该字符的字符串表示 (`__repr__()`) 与打印出来的表示有何不同？

提交内容：一句话回答。

(c) 当这个字符出现在文本中时，会发生什么？你可以在 Python 解释器中尝试以下操作，看看结果是否符合预期：

```
>>> chr(0)
>>> print(chr(0))
```

```
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

提交内容：一句话回答。

2.2 Unicode 编码

虽然 Unicode 标准定义了字符到码点（整数）的映射，但直接在 Unicode 码点上训练分词器并不实际，因为词表会大得难以处理（约 15 万项），而且非常稀疏（许多字符十分罕见）。因此，我们改用 Unicode 编码，将 Unicode 字符转换为字节序列。Unicode 标准本身定义了三种编码：UTF-8、UTF-16 和 UTF-32，其中 UTF-8 是互联网上占主导地位的编码，超过 98% 的网页使用它。

要将 Unicode 字符串编码为 UTF-8，可以使用 Python 的 `encode()` 函数。要访问 Python `bytes` 对象底层的字节值，可以对其进行迭代（例如调用 `list()`）。最后，可以使用 `decode()` 函数将 UTF-8 字节串解码为 Unicode 字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # 获取编码后字符串的字节值（0 到 255 之间的整数）。
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129, 161, 227, 129, 175, 33]
>>> # 一个字节并不一定对应一个 Unicode 字符!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

将 Unicode 码点转换为字节序列（例如使用 UTF-8 编码），实质上就是把码点序列（具有 159,801 个有效取值的 21 位整数）转换为字节值序列（0 到 255 范围内的整数）。这个长度为 256 的字节词表要容易处理得多。使用字节级分词时，不必担心词表外词元，因为我们知道，任何输入文本都能表示为由 0 到 255 之间的整数组成的序列。

问题 (unicode2)：Unicode 编码 (3 分)

(a) 与 UTF-16 或 UTF-32 相比，在 UTF-8 编码的字节上训练分词器有哪些优势？比较这些编码对于不同输入字符串的输出，可能会有所帮助。

提交内容：一至两句话回答。

(b) 考虑下面这个意图将 UTF-8 字节串解码为 Unicode 字符串的（错误）函数。为什么它不正确？请给出一个会产生错误结果的输入字节串示例。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

提交内容：一个使 `decode_utf8_bytes_to_str_wrong` 产生错误输出的输入字节串示例，并用一句话解释该函数为什么不正确。

(c) 给出一个无法解码为任何 Unicode 字符的双字节序列。

提交内容：一个示例，并用一句话解释。

2.3 子词分词

虽然字节级分词能够缓解词级分词器面临的词表外问题，但将文本切分为字节会得到极长的输入序列。这会拖慢模型训练：一个包含 10 个词的句子，在词级语言模型中可能只有 10 个词元，而在字符级模型中则可能长达 50 个或更多词元，具体取决于词的长度。处理这些更长的序列，需要模型在每一步进行更多计算。此外，在字节序列上进行语言建模也比较困难，因为更长的输入序列会在数据中产生长程依赖。

子词分词介于词级分词与字节级分词之间。注意，字节级分词器的词表有 256 项（字节值为 0 到 255）。子词分词器通过增大词表，换取对输入字节序列更好的压缩。例如，如果字节序列 `b'the'` 经常出现在原始文本训练数据中，为它分配一个词表条目，就可以将这个原本长为 3 个词元的序列缩减为单个词元。

我们如何选择要加入词表的子词单元？R. Sennrich 等 [3] 提出使用字节对编码（BPE；P. Gage [5]），这是一种压缩算法，会迭代地用一个新的、尚未使用的索引替换（“合并”）最频繁的字节对。注意，该算法向词表中添加子词词元，目的是尽可能压缩输入序列——如果某个词在输入文本中出现得足够频繁，它最终就会用一个子词单元表示。

使用 BPE 构建词表的子词分词器，通常称为 BPE 分词器。本作业中，我们将实现字节级 BPE 分词器，其词表项是字节或合并后的字节序列，从而兼顾词表外处理能力和可控的输入序列长度。构建 BPE 分词器词表的过程称为“训练”BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器的训练过程包含三个主要步骤。

初始化词表

分词器词表是字节串词元与整数 ID 之间的一一映射。由于我们训练的是字节级 BPE 分词器，初始词表就是所有字节的集合。字节有 256 种可能取值，因此初始词表大小为 256。

预分词

有了词表后，原则上可以统计文本中字节相邻出现的频率，并从最频繁的字节对开始进行合并。然而，这样的计算开销很大，因为每进行一次合并，都必须完整遍历一遍语料库。此外，

直接在整个语料库中合并字节，可能生成一些仅在标点符号上有所不同的词元（例如 `dog!` 与 `dog.`）。这些词元的 ID 会完全不同，尽管它们很可能在语义上高度相似，因为两者仅在标点上有差别。

为避免这一点，我们先对语料库进行预分词。你可以将它理解为对语料库的一种粗粒度分词，帮助我们统计字符对出现的频率。例如，单词 `'text'` 可能是一个出现 10 次的预词元。那么，在统计字符“t”和“e”相邻出现的频率时，我们看到“text”中的“t”和“e”相邻，就可以直接将计数增加 10，而无须遍历语料库。由于我们训练的是字节级 BPE 模型，每个预词元都表示为 UTF-8 字节序列。

R. Sennrich 等 [3] 的原始 BPE 实现仅通过按空白拆分来预分词（即 `s.split(" ")`）。这种方法仍可见于基于 SentencePiece 的分词器中，例如 Llama 1 和 Llama 2 的分词器。

大多数现代分词器使用基于正则表达式的预分词器，这一做法源自 GPT-2；A. Radford 等 [6]。我们将使用原始正则表达式的一个略为简洁的版本，取自：<https://github.com/openai/tiktoken/pull/234/files>

```
>>> PAT = r'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[\s\p{L}\p{N}]+\s+(?!\S)|\s+''''
```

可以交互式地使用这个预分词器拆分一些文本，以便更好地理解其行为：

```
>>> # 需要安装 `regex` 包
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

不过，在你的代码中使用它时，应使用 `re.finditer`，从而在构建预词元到出现次数的映射时，避免额外存储预分词后的词。

计算 BPE 合并

现在，我们已经将输入文本转换为预词元，并将每个预词元表示为 UTF-8 字节序列，可以开始计算 BPE 合并（即训练 BPE 分词器）。总体而言，BPE 算法会迭代统计每个字节对，找出频率最高的一对（“A”，“B”）。然后，将这一最频繁字节对的每次出现都合并，即替换为新词元“AB”。这个合并后的新词元会加入词表；因此，BPE 训练后的最终词表大小等于初始词表大小（这里为 256）加上训练过程中执行的 BPE 合并次数。为提高训练效率，我们不考虑跨越预词元边界的字节对。² 计算合并时，如果多个字节对频率相同，应确定性地选择字典序较大的字节对。例如，若（“A”，“B”）、（“A”，“C”）、（“B”，“ZZ”）和（“BA”，“A”）的频率并列最高，则合并（“BA”，“A”）：

```
>>> max([(("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A"))],
('BA', 'A'))
```

特殊词元

某些字符串（例如 `<|endoftext|>`）经常被用来编码元数据（例如文档边界）。编码文本时，通常希望将某些字符串作为“特殊词元”，使它们

² 注意，R. Sennrich 等 [3] 对 BPE 的原始描述要求加入词尾词元。在训练字节级 BPE 模型时，我们不添加词尾词元，因为所有字节（包括空白与标点）都已包含在模型词表中。由于我们显式表示了空格与标点，学习到的 BPE 合并自然会体现这些词边界。

永远不会被拆成多个词元，即始终保留为单个词元。例如，表示序列结束的字符串 `<|endoftext|>` 应始终保留为单个词元（一个整数 ID），这样我们才能知道何时停止语言模型的生成。这些特殊词元必须加入词表，从而具有对应的固定词元 ID。

R. Sennrich 等 [3] 的算法 1 给出了一个效率较低的 BPE 分词器训练实现，基本上遵循了上面概述的步骤。作为第一个练习，实现并测试这个函数，可能有助于检查你的理解。

示例 (bpe_example): BPE 训练示例

下面是取自 R. Sennrich 等 [3] 的一个简化示例。考虑包含以下文本的语料库：

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

词表中还包含一个特殊词元 `<|endoftext|>`。

词表

我们使用特殊词元 `<|endoftext|>` 和全部 256 个字节值初始化词表。

预分词

为了简化并聚焦于合并过程，假设本例的预分词仅按空白拆分。预分词并统计后，得到如下频数表：

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

可将其方便地表示为 `dict[tuple[bytes, ...], int]`，例如 `{(l,o,w): 5, ...}`。注意，在 Python 中，即使是单个字节也是 `bytes` 对象。Python 没有用于表示单个字节的 `byte` 类型，正如它没有用于表示单个字符的 `char` 类型一样。

合并

首先，检查每个相邻字节对，并累加它们所在单词的频数：

```
{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}
```

字节对 `('e', 's')` 与 `('s', 't')` 并列，因此选择字典序较大的 `('s', 't')`。在预词元中进行合并后得到：

```
{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}
```

第二轮中，`(e, st)` 是最常见的字节对，计数为 9；合并后得到：

```
{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}
```

继续执行，最终得到的合并序列为：

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e',
 ↪ 'lowe r']
```

如果只执行 6 次合并，则合并列表为：

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']
```

词表元素则为：

```
[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]
```

使用这个词表与这组合并，单词 `newest` 会被切分为 `[ne, west]`。

2.5 BPE 分词器训练实验

接下来，在 TinyStories 数据集上训练字节级 BPE 分词器。查找/下载数据集的说明见第 1 节。开始之前，建议先浏览 TinyStories 数据集，了解数据内容。

并行化预分词

你会发现，预分词步骤是一个主要瓶颈。可以使用内置的 `multiprocessing` 库并行化代码，从而加快预分词。具体而言，对于并行预分词实现，我们建议将语料库切分为若干块，并确保每个分块边界位于某个特殊词元的起始位置。你可以原样使用下列链接中的起始代码获取分块边界，再据此将工作分配给不同进程：https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py 这种分块始终有效，因为我们本来就不希望跨文档边界合并。对于本作业，始终可以按这种方式切分，不必考虑收到一个完全不包含 `<|endoftext|>` 的超大语料库这一边界情况。

在预分词前移除特殊词元

在使用正则表达式模式进行预分词（通过 `re.finditer`）之前，应移除语料库中的所有特殊词元；若使用并行实现，则对每个分块执行这一操作。务必以特殊词元为分隔符切分，确保不会跨越它们划定的文本边界发生合并。例如，若某个语料库或分块为 `[Doc 1]<|endoftext|>[Doc 2]`，则应在特殊词元 `<|endoftext|>` 处拆分，分别对 `[Doc 1]` 和 `[Doc 2]` 预分词，使合并无法跨越文档边界。换言之，特殊词元在训练中定义了硬性的切分边界，但它们本身不应计入合并统计。可使用 `re.split`，并将 `"|".join(special_tokens)` 作为分隔符来实现；要谨慎使用 `re.escape`，因为特殊词元中可能含有 `|`。测试 `test_train_bpe_special_tokens` 会检查这一点。

优化合并步骤

上面简化示例中的朴素 BPE 训练实现很慢，因为每次合并都要遍历所有字节对，找出频率最高的一对。然而，每次合并后，只有与被合并字节对重叠的那些字节对的计数会发生变化。因此，可以为所有字节对的计数建立索引，并增量更新这些计数，而不是显式遍历每个字节对来统计频率，从而加快 BPE 训练。这个缓存过程能够带来显著加速；不过，需要注意，BPE 训练中的合并部分无法在 Python 中并行化。

低资源提示：性能剖析

应使用 `cProfile` 或 `py-spy` 等性能剖析工具，找出实现中的瓶颈，并集中优化这些部分。

低资源提示：“缩小规模”

不要一开始就在完整 TinyStories 数据集上训练分词器。我们建议先在数据的一小部分——“调试数据集”——上训练。例如，可以改用 TinyStories 验证集，其中包含 2.2 万篇文档，而不是 212 万篇。这体现了一种通用策略：只要可能，就通过缩小规模加快开发，例如，

使用更小的数据集、更小的模型等。调试数据集的规模或超参数配置需要仔细斟酌：调试集应足够大，使其与完整配置具有相同的瓶颈，从而让你做出的优化可以推广；但也不能太大，以至于一次运行就耗时很久。

问题 (train_bpe): BPE 分词器训练 (15 分)

提交内容：编写一个函数，给定输入文本文件的路径，训练一个（字节级）BPE 分词器。你的 BPE 训练函数应至少处理以下输入参数：

输入

`input_path: str`: 包含 BPE 分词器训练数据的文本文件路径。

`vocab_size: int`: 一个正整数，定义最终词表大小的上限，包括初始字节词表、合并产生的词表项以及全部特殊词元。

`special_tokens: list[str]`: 需要添加到词表中的字符串列表。训练时，将它们视为硬边界，阻止跨越其所在范围的合并，但不要将它们计入合并统计。

BPE 训练函数应返回生成的词表和合并列表：

输出

`vocab: dict[int, bytes]`: 分词器词表，将 `int`（词表中的词元 ID）映射到 `bytes`（词元的字节）。

`merges: list[tuple[bytes, bytes]]`: 训练产生的 BPE 合并列表。每个列表项都是字节串元组 (`<token1>`, `<token2>`)，表示 `<token1>` 与 `<token2>` 被合并。合并列表应按创建顺序排列。

要用我们提供的测试检查 BPE 训练函数，首先需要实现测试适配器 `adapters.run_train_bpe`，然后运行：

```
uv run pytest tests/test_train_bpe.py
```

你的实现应通过所有测试。你也可以选择使用系统编程语言实现训练方法中的关键部分，例如 C++（可考虑 `cppyy` 或 `nanobind`）或 Rust（使用 `PyO3`）；但这可能需要投入大量时间。若这样做，请注意哪些操作需要复制数据、哪些操作可以直接读取 Python 内存，并务必提供构建说明，或者确保仅通过 `pyproject.toml` 就能完成构建。另外，大多数正则表达式引擎对 GPT-2 正则表达式的支持并不好，而多数支持它的引擎又太慢。我们验证过，`Oniguruma` 的速度尚可，并支持负向前瞻；不过 Python 的 `regex` 包甚至可能更快。

问题 (train_bpe_tinystories): 在 TinyStories 上训练 BPE (2 分)

(a) 在 TinyStories 数据集上训练字节级 BPE 分词器，最大词表大小设为 10,000。务必将 TinyStories 的特殊词元 `<|endoftext|>` 加入词表。将生成的词表和合并列表序列化保存到磁盘，以便进一步检查。训练用了多少时间和内存？词表中最长的词元是什么？它合理吗？

资源要求：不超过 30 分钟（不使用 GPU），不超过 30 GB 内存。

提示：在预分词阶段使用 `multiprocessing`，并利用以下两个事实，应该能够将 BPE 训练时间降到 2 分钟以内：

(a) 数据文件中使用 `<|endoftext|>` 词元分隔文档。

(b) 在应用 BPE 合并之前，会将 `<|endoftext|>` 词元作为特殊情况处理。

提交内容：一至两句话回答。

(b) 对代码进行性能剖析。分词器训练流程中，哪一部分最耗时？

提交内容：一至两句话回答。

接下来，尝试在 OpenWebText 数据集上训练字节级 BPE 分词器。和之前一样，我们建议先浏览数据集，以便更好地理解其中的内容。

问题 (train_bpe_expts_owt)：在 OpenWebText 上训练 BPE (2 分)

(a) 在 OpenWebText 数据集上训练字节级 BPE 分词器，最大词表大小设为 32,000。将生成的词表和合并列表序列化保存到磁盘，以便进一步检查。词表中最长的词元是什么？它合理吗？

资源要求：不超过 12 小时（不使用 GPU），不超过 100 GB 内存。

提交内容：一至两句话回答。

(b) 比较在 TinyStories 上训练的分词器与在 OpenWebText 上训练的分词器，说明它们的异同。

提交内容：一至两句话回答。

2.6 BPE 分词器：编码与解码

在作业的上一部分，我们实现了一个函数，用输入文本训练 BPE 分词器，得到分词器词表和 BPE 合并列表。现在，我们将实现一个 BPE 分词器，加载给定的词表和合并列表，并使用它们在文本与词元 ID 之间进行编码和解码。

2.6.1 文本编码

BPE 对文本进行编码的过程与训练 BPE 词表的过程相对应，主要包括以下几个步骤。**步骤 1：预分词。**首先，与 BPE 训练时一样，对序列进行预分词，并将每个预词元表示为 UTF-8 字节序列。之后，我们会在每个预词元内部，将字节合并为词表中的元素；每个预词元独立处理，不跨越预词元边界合并。**步骤 2：应用合并。**取出 BPE 训练时产生的词表元素合并序列，并按相同的创建顺序，将这些合并应用到预词元上。

示例 (bpe_encoding)：BPE 编码示例

例如，假设输入字符串是 'the cat ate'，词表为：

```
{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9:
  ↳ b'the', 10: b' at'}
```

学习到的合并列表为：

```
[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]
```

首先，预分词器会将该字符串切分为 ['the', ' cat', ' ate']。然后，分别检查每个预词元并应用 BPE 合并。

第一个预词元 'the' 最初表示为 [b't', b'h', b'e']。查看合并列表，找到第一个可应用的合并 (b't', b'h')，将预词元转换为 [b'th', b'e']。然后回到合并列表，找到下一个可应用的合并 (b'th', b'e')，将预词元转换为 [b'the']。最后，再次查看合并列表，发现已经没有可应用于该字符串的合并，因为整个预词元已合并为单个词元，因此 BPE 合并过程结束。对应的整数序列为 [9]。

对剩余预词元重复这一过程。预词元 ' cat' 在应用 BPE 合并后表示为 [b' c', b'a', b't']，转换为整数序列 [7, 1, 5]。最后一个预词元 ' ate' 在合并后为 [b' at', b'e']，转换为整数序列 [10, 3]。因此，输入字符串的最终编码结果为 [9, 7, 1, 5, 10, 3]。

特殊词元

编码文本时，分词器应能够正确处理用户定义的特殊词元；这些特殊词元在构造分词器时提供。

内存方面的考虑

假设要对一个无法整体放入内存的大型文本文件进行分词。为高效处理这个大文件或其他数据流，需要将其划分为大小可控的分块，依次处理各块，使内存复杂度保持常数，而不是随文本大小线性增长。这样做时，必须确保没有词元跨越分块边界，否则得到的分词结果将与把整个序列一次性加载到内存后进行分词的朴素方法不同。

2.6.2 文本解码

要将整数词元 ID 序列解码回原始文本，只需在词表中查找每个 ID 对应的条目（字节序列），将它们拼接，再把字节解码成 Unicode 字符串。注意，输入 ID 不保证一定映射为有效的 Unicode 字符串，因为用户可能输入任意整数 ID 序列。如果输入词元 ID 无法产生有效的 Unicode 字符串，应将格式错误的字节替换为官方 Unicode 替换字符 U+FFFD。³bytes.decode 的 errors 参数用于控制 Unicode 解码错误的处理方式；使用 errors='replace' 会自动将格式错误的字节替换为该替换标记。

问题 (tokenizer)：实现分词器 (15 分)

提交内容：实现一个 Tokenizer 类，给定词表和合并列表，将文本编码为整数 ID，并将整数 ID 解码为文本。分词器还应支持用户提供的特殊词元；若它们尚未出现在词表中，就将它们追加到词表。我们建议采用以下接口：

³ 关于 Unicode 替换字符的更多信息，见 [https://en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character)。

```
def __init__(self, vocab, merges, special_tokens=None)
```

根据给定词表、合并列表以及可选的特殊词元列表构造分词器。此函数应接受以下参数：

```
vocab: dict[int, bytes]
merges: list[tuple[bytes, bytes]]
special_tokens: list[str] | None = None
```

```
def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)
```

类方法：从序列化的词表和合并列表，以及可选的特殊词元列表，构造并返回一个 `Tokenizer`。序列化格式应与你的 BPE 训练代码输出相同。该方法应接受以下额外参数：

```
vocab_filepath: str
merges_filepath: str
special_tokens: list[str] | None = None
```

```
def encode(self, text: str) -> list[int]
```

将输入文本编码为词元 ID 序列。

```
def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]
```

给定字符串的可迭代对象（例如 Python 文件句柄），返回一个惰性地产生词元 ID 的生成器。对于无法直接加载到内存的大文件，这项功能是实现内存高效分词所必需的。

```
def decode(self, ids: list[int]) -> str
```

将词元 ID 序列解码为文本。

要用我们提供的测试检查 `Tokenizer`，首先实现测试适配器 `adapters.get_tokenizer`，然后运行 `uv run pytest tests/test_tokenizer.py`。你的实现应通过所有测试。

2.7 实验

问题 (tokenizer_experiments)：分词器实验 (4 分)

(a) 从 `TinyStories` 和 `OpenWebText` 中抽样 10 篇文档。使用之前训练好的 `TinyStories` 分词器和 `OpenWebText` 分词器，词表大小分别为 10K 和 32K，将抽样文档编码为整数 ID。每个分词器的压缩比是多少（字节／词元）？

提交内容：一至两句话回答。

(b) 如果使用 `TinyStories` 分词器对 `OpenWebText` 样本进行分词，会发生什么？比较压缩比，和／或定性描述结果。

提交内容：一至两句话回答。

(c) 估算分词器的吞吐量，例如以字节／秒为单位。对 `Pile` 数据集（825 GB 文本）进行分词需要多长时间？

提交内容：一至两句话回答。

(d) 使用 `TinyStories` 和 `OpenWebText` 分词器，将各自的训练集和开发集编码为整数词元 ID 序列。之后，我们将使用它们训练

语言模型。建议将词元 ID 序列化为数据类型为 `uint16` 的 NumPy 数组。为什么 `uint16` 是合适的选择？

提交内容：一至两句话回答。

3 Transformer 语言模型架构

语言模型的输入是成批的整数词元 ID 序列，即形状为 $(\text{batch_size}, \text{sequence_length})$ 的 `torch.Tensor`；输出是词表上的成批归一化概率分布，即形状为 $(\text{batch_size}, \text{sequence_length}, \text{vocab_size})$ 的 PyTorch 张量，其中每个输入词元对应一个关于下一个词的预测分布。训练语言模型时，我们使用这些下一个词的预测，计算真实下一个词与预测下一个词之间的交叉熵损失。推理时从语言模型生成文本，则取最后一个时间步，也就是序列最后一项的下一个词预测分布，生成序列中的下一个词元（例如选择概率最高的词元，或从分布中采样），将生成的词元加入输入序列，然后重复这一过程。

在本部分中，你将从零构建这个 Transformer 语言模型。我们先给出模型的高层次描述，再逐步详细介绍各个组件。

3.1 Transformer 语言模型

给定词元 ID 序列，Transformer 语言模型使用输入嵌入将词元 ID 转换为稠密向量，将嵌入后的词元依次送入 `num_layers` 个 Transformer 块，再应用一个可学习的线性投影（“输出嵌入”或“LM head”），得到预测下一个词元的 logits。示意图见图 1。

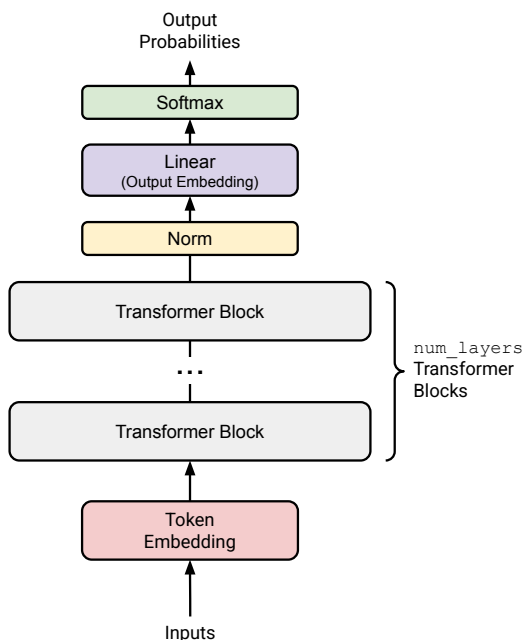


图 1: Transformer 语言模型概览。

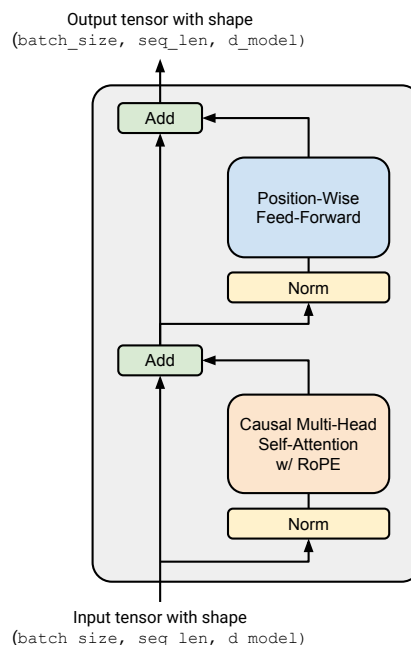


图 2: 前置归一化 Transformer 块。

词元嵌入

第一步，Transformer 将成批的词元 ID 序列嵌入为向量序列，其中包含词元身份信息，即图 1 中的红色模块。

更具体地说，给定词元 ID 序列，Transformer 语言模型使用词元嵌入层生成向量序列。每个嵌入层接收形状为 $(\text{batch_size}, \text{sequence_length})$ 的整数张量，并生成形状为 $(\text{batch_size}, \text{sequence_length}, \text{d_model})$ 的向量序列。

前置归一化 Transformer 块

嵌入之后，激活值会经过若干结构相同的神经网络层处理。标准的仅解码器 Transformer 语言模型由 num_layers 个相同的层组成，通常称为 Transformer “块”。每个 Transformer 块接收形状为 $(\text{batch_size}, \text{sequence_length}, \text{d_model})$ 的输入，并返回相同形状的输出。每个块通过自注意力聚合整个序列的信息，再通过前馈层进行非线性变换。

经过 num_layers 个 Transformer 块后，我们会取最终激活值，将其转换为词表上的分布。

我们将实现“前置归一化”（pre-norm）Transformer 块，详见第 3.4 节。该结构还要求在最后一个 Transformer 块之后额外进行层归一化（下文详述），以确保输出具有适当的尺度。

归一化之后，我们使用一个标准的可学习线性变换，将 Transformer 块的输出转换为预测下一个词元的 logits，例如参见 A. Radford 等 [7] 的公式 2。

3.2 说明：批处理、Einsum 与高效计算

在整个 Transformer 中，我们会将相同计算应用到许多具有类似批维度的输入上。下面是几个例子：

- **批中的元素**：对批中的每个元素应用相同的 Transformer 前向运算。
- **序列长度**：RMSNorm 和前馈网络等“逐位置”操作，对序列中的每个位置执行相同运算。
- **注意力头**：在“多头”注意力操作中，对各注意力头批量执行注意力运算。

如果有一种方便的表达方式，既能充分利用 GPU，又易于阅读和理解，就会很有帮助。许多 PyTorch 操作可以接受张量前面额外的“类批维度”，并高效地沿这些维度重复或广播该操作。

例如，假设要进行逐位置的批量运算。我们有一个形状为 $(\text{batch_size}, \text{sequence_length}, \text{d_model})$ 的“数据张量” D ，希望让它与形状为 $(\text{d_model}, \text{d_model})$ 的矩阵 A 进行批量向量—矩阵乘法。在这种情况下， $D @ A$ 会执行批量矩阵乘法，这是 PyTorch 中一种高效的基本操作，其中 $(\text{batch_size}, \text{sequence_length})$ 维度都作为批维度处理。

因此，最好假设你的函数可能接收到额外的类批维度，并将这些维度放在 PyTorch 张量形状的最前面。为了使张量能够按这种方式进行批处理，可能需要多次使用 `view`、`reshape` 和 `transpose` 来调整形状。这有些麻烦，也常常使代码的作用和张量的形状变得难以看懂。

一种更方便的选择是在 `torch.einsum` 中使用爱因斯坦求和记法 (`einsum`)，或者使用 `einops`、`einx` 等与框架无关的库。两个关键操作是 `einsum` 和 `rearrange`：前者可以对输入张量的任意维度进行张量缩并，后者可以对任意维度进行重排、合并和拆分。事实表明，机器学习中的几乎所有操作，都可以表示为维度变换与张量缩并的某种组合，再偶尔加入非线性函数，通常是逐元素的。因此，使用 `einsum` 记法可以让大量代码更易读、更灵活。

我们强烈建议在本课程中学习并使用 `einsum` 记法。从未接触过它的同学应使用 `einops` (文档在这里)；已经熟练掌握 `einops` 的同学，应学习更通用的 `einx` (文档在这里)。⁴ 这两个包都已安装在我们提供的环境中。

下面给出几个 `einsum` 记法的使用示例。这些示例是 `einops` 文档的补充，请先阅读该文档。

示例 (einstein_example1)：使用 `einops.einsum` 进行批量矩阵乘法

```
import torch
from einops import rearrange, einsum

## 基本实现
Y = D @ A.T
# 很难看出输入与输出的形状及其含义。
# D 和 A 可以具有哪些形状？是否有某些形状会导致意外行为？

## Einsum 自带说明性，且更加稳健
#           D               A               ->           Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## 或者使用批处理版本：D 可以有任意前导维度，但 A 的形状受到约束。
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

示例 (einstein_example2)：使用 `einops.rearrange` 进行广播运算

给定一批图像，希望根据一些缩放系数，为每张图像生成 10 个亮度降低的版本：

```
images = torch.randn(64, 128, 128, 3) # (批大小, 高度, 宽度, 通道)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## 调整形状并相乘
dim_value = rearrange(dim_by, "dim_value -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## 或者一步完成：
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

⁴ 值得注意的是，尽管 `einops` 得到了广泛支持，`einx` 还没有经过同等充分的实际检验。如果遇到 `einx` 的限制或错误，可以退回使用 `einops`，配合更多普通 PyTorch 操作。

示例 (einstein_example3): 使用 einops.rearrange 混合像素

假设有一批图像, 表示为形状为 (batch, height, width, channel) 的张量, 希望对图像的全部像素进行线性变换, 但每个通道应独立执行该变换。线性变换表示为形状为 (height * width, height * width) 的矩阵 B 。

```
channels_last = torch.randn(64, 32, 32, 3) # (批大小, 高度, 宽度, 通道)
B = torch.randn(32*32, 32*32)

## 重排图像张量, 以便混合所有像素
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

也可以改用 einops:

```
height = width = 32
## rearrange 替代笨重的 torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)
```

或者, 更大胆一些: 使用 einx.dot 一步完成, 它是 einx 中对应 einops.einsum 的操作。

```
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

第一种实现可以通过在代码前后添加注释, 标明输入和输出形状来改进; 但这既繁琐又容易出错。使用 einsum 记法时, 文档就是实现!

einsum 记法可以处理任意输入批维度, 同时还具有自说明性这一关键优势。使用 einsum 记法的代码能更清楚地展示输入和输出张量的相关形状。

对于其余张量，可以考虑使用 Tensor 类型提示，例如使用 jaxtyping 库，它并非仅适用于 JAX。

我们将在作业 2 中进一步讨论使用 einsum 记法对性能的影响；目前只需知道，它们几乎总是优于替代写法！

3.2.1 数学记法与内存顺序

许多机器学习论文采用行向量记法，得到的表示方式与 NumPy 和 PyTorch 默认使用的行主序内存布局很契合。使用行向量时，线性变换写作：

$$y = xW^{\top}, \quad (1)$$

其中， $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 按行主序存储， $x \in \mathbb{R}^{1 \times d_{\text{in}}}$ 是行向量。注意，这样可以通过增大 x 的最外层维度来批处理输入，也就是说，可以将向量输入 x 替换为矩阵输入 $X \in \mathbb{R}^{\text{batch} \times d_{\text{in}}}$ 。

在线性代数中，更常使用列向量，此时线性变换写作：

$$y = Wx, \quad (2)$$

其中， $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 按行主序存储， $x \in \mathbb{R}^{d_{\text{in}}}$ 是列向量。在这种情况下批处理输入时， x 的批维度必须放在最后，因此需要将 x 替换为矩阵 $\tilde{X} \in \mathbb{R}^{d_{\text{in}} \times \text{batch}}$ 。**本作业的数学记法将主要使用列向量**，因为数学中通常遵循这一约定。请记住，由于 PyTorch 使用行主序内存布局，如果采用普通矩阵乘法记法，就需要像公式 1 的行向量约定那样，在应用矩阵时进行转置。如果使用 einsum 进行线性代数运算，只要正确标记各个轴，就不必担心这一点。顺便一提，Matlab、Julia 和 Fortran 等其他语言／线性代数软件包都使用列主序内存布局，因此批维度放在最后；而 Python 及其相关软件包则采用了 C 语言的行主序标准。

3.3 基本构建模块：Linear 与 Embedding 模块

3.3.1 参数初始化

为了有效训练神经网络，往往需要仔细初始化模型参数——不良初始化可能导致梯度消失或梯度爆炸等不希望出现的现象。前置归一化 Transformer 对初始化格外稳健，但初始化仍可能显著影响训练速度与收敛。由于本作业已经很长，我们将细节留到作业 3；这里先给出一些近似的初始化设置，通常在多数情况下都能取得良好效果。现在请使用：

- 线性层权重： $\mathcal{N}\left(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}}\right)$ ，截断范围为 $[-3\sigma, 3\sigma]$ 。
- 嵌入： $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ ，截断范围为 $[-3, 3]$ 。
- RMSNorm：1。

应使用 `torch.nn.init.trunc_normal_` 初始化服从截断正态分布的权重。

3.3.2 Linear 模块

线性层是 Transformer 以及一般神经网络的基本构建模块。首先，实现你自己的 Linear 类，使其继承 `torch.nn.Module`，并执行线性变换：

$$y = Wx. \quad (3)$$

注意，遵循大多数现代大语言模型的做法，我们不包含偏置项。

问题 (linear)：实现线性模块 (1 分)

提交内容：实现一个继承 `torch.nn.Module` 的 `Linear` 类，用于执行线性变换。除不包含 `bias` 参数或偏置参数张量外，你的实现应遵循 PyTorch 内置 `nn.Linear` 模块的接口。我们建议采用以下接口：

```
def __init__(self, in_features, out_features, device=None, dtype=None)
```

构造线性变换模块。此函数应接受以下参数：

`in_features: int`：输入的最后一个维度。

`out_features: int`：输出的最后一个维度。

`device: torch.device | None = None`：存储参数的设备。

`dtype: torch.dtype | None = None`：参数的数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

对输入应用线性变换。

务必做到：

- 继承 `nn.Module`。
- 调用父类构造函数。
- 按 W 而不是 W^T 的形式构造和存储参数，并将其放入 `nn.Parameter`。
- 当然，不要使用 `nn.Linear` 或 `nn.functional.linear`。

初始化时，使用上面给出的设置，通过 `torch.nn.init.trunc_normal_` 初始化权重。

要测试 `Linear` 模块，实现测试适配器 `adapters.run_linear`。该适配器应将给定权重加载到你的 `Linear` 模块中，可以使用 `Module.load_state_dict` 完成。然后运行 `uv run pytest -k test_linear`。

3.3.3 Embedding 模块

如上所述，Transformer 的第一层是嵌入层，将整数词元 ID 映射到维度为 `d_model` 的向量空间。我们将实现一个自定义 `Embedding` 类，使其继承 `torch.nn.Module`，因此不能使用 `nn.Embedding`。其 `forward` 方法应使用形状为 `(batch_size, sequence_length)`、包含词元 ID 的 `torch.LongTensor`，对形状为 `(vocab_size, d_model)` 的嵌入矩阵进行索引，为每个词元 ID 选出对应的嵌入向量。

问题 (embedding)：实现嵌入模块 (1 分)

提交内容：实现继承 `torch.nn.Module` 的 `Embedding` 类，执行嵌入查找。你的实现应遵循 PyTorch 内置 `nn.Embedding` 模块的接口。我们建议采用以下接口：

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

构造嵌入模块。此函数应接受以下参数：

`num_embeddings: int`: 词表大小。

`embedding_dim: int`: 嵌入向量维度，即 d_{model} 。

`device: torch.device | None = None`: 存储参数的设备。

`dtype: torch.dtype | None = None`: 参数的数据类型。

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

查找给定词元 ID 对应的嵌入向量。

务必做到：

- 继承 `nn.Module`。
- 调用父类构造函数。
- 将嵌入矩阵初始化为 `nn.Parameter`。
- 存储嵌入矩阵时，将 `d_model` 放在最后一个维度。
- 当然，不要使用 `nn.Embedding` 或 `nn.functional.embedding`。

同样，采用上面给出的初始化设置，并使用 `torch.nn.init.trunc_normal_` 初始化权重。

要测试实现，完成测试适配器 `adapters.run_embedding`，然后运行 `uv run pytest -k test_embedding`。

3.4 前置归一化 Transformer 块

每个 Transformer 块包含两个子层：多头自注意力机制和逐位置前馈网络（A. Vaswani 等 [2017]，第 3.1 节）。

原始 Transformer 论文在每个子层外使用残差连接，然后进行层归一化。由于层归一化应用于子层输出，这种架构通常称为“后置归一化”（post-norm）Transformer。然而，多项研究发现，将层归一化从每个子层的输出端移到输入端，并在最后一个 Transformer 块之后再增加一次层归一化，可以提高 Transformer 的训练稳定性 [T. Q. Nguyen 等, 2019; R. Xiong 等, 2020]。这一“前置归一化”（pre-norm）Transformer 块的图示见图 2。随后，每个 Transformer 块子层的输出通过残差连接加到该子层输入上（A. Vaswani 等 [8]，第 5.4 节）。对前置归一化的一种直观理解是：从输入嵌入一直到 Transformer 的最终输出，存在一条不经过任何归一化的干净“残差流”，据称这有助于改善梯度流动。如今，前置归一化 Transformer 已成为语言模型的标准选择，例如 GPT-3、LLaMA 和 PaLM，因此我们将实现这一变体。下面会逐一介绍前置归一化 Transformer 块中的组件，并依次实现它们。

3.4.1 均方根层归一化

A. Vaswani 等 [8] 的原始 Transformer 实现使用层归一化 [J. L. Ba 等, 2016] 来归一化激活值。遵循 H. Touvron 等 [12]，我们使用均方根层归一化（RMSNorm；B. Zhang 等 [13]，公式 4）。给定激活向量 $a \in \mathbb{R}^{d_{\text{model}}}$ ，RMSNorm 按如下方式重新缩放每个激活值 a_i ：

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \quad (4)$$

其中, $\text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2} + \varepsilon$ 。这里, g_i 是可学习的“增益”参数, 共有 d_{model} 个; ε 是一个超参数, 通常固定为 $1e-5$ 。

应将输入提升为 `torch.float32`, 防止对输入求平方时发生溢出。总体而言, `forward` 方法应如下所示:

```
in_dtype = x.dtype
x = x.to(torch.float32)
# 在这里编写执行 RMSNorm 的代码
...
result = ...
# 以原始数据类型返回结果
return result.to(in_dtype)
```

问题 (rmsnorm): 均方根层归一化 (1 分)

提交内容: 将 `RMSNorm` 实现为一个 `torch.nn.Module`。建议采用以下接口:

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
```

构造 `RMSNorm` 模块。此函数应接受以下参数:

`d_model: int`: 模型的隐藏维度。

`eps: float = 1e-5`: 用于数值稳定性的 `epsilon` 值。

`device: torch.device | None = None`: 存储参数的设备。

`dtype: torch.dtype | None = None`: 参数的数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

处理形状为 `(batch_size, sequence_length, d_model)` 的输入张量, 返回相同形状的张量。

注意: 如上所述, 记得在归一化之前将输入提升为 `torch.float32`, 并在之后转换回原来的数据类型。

要测试实现, 完成测试适配器 `adapters.run_rmsnorm`, 然后运行 `uv run pytest -k test_rmsnorm`。

3.4.2 逐位置前馈网络

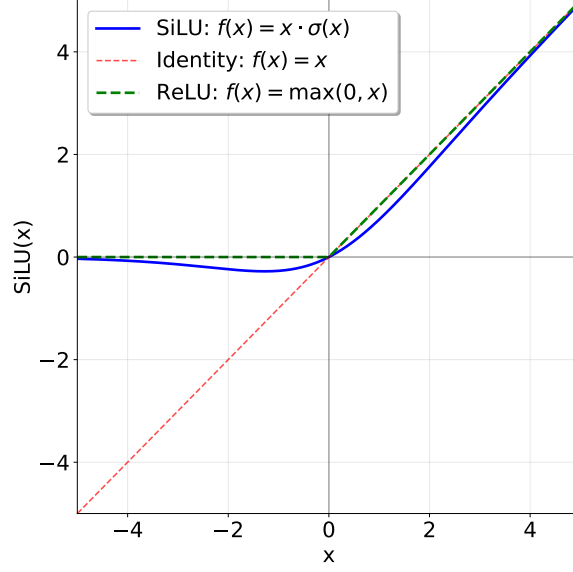


图 3: SiLU（也称 Swish）与 ReLU 激活函数的比较。

在原始 Transformer 论文中 (A. Vaswani 等 [8], 第 3.3 节), Transformer 的前馈网络由两个线性变换组成, 中间使用 ReLU 激活函数, $\text{ReLU}(x) = \max(0, x)$ 。在该原始架构中, 前馈网络中间层的维度通常是输入维度的 4 倍。

然而, 相较于原始设计, 现代语言模型往往有两项主要修改: 采用另一种激活函数, 并引入门控机制。具体而言, 我们将实现 Llama 3 [A. Grattafiori 等, 2024] 和 Qwen 2.5 [A. Yang 等, 2024] 等大语言模型采用的 “SwiGLU” 激活函数, 它将 SiLU (通常称为 Swish) 激活与一种称为门控线性单元 (GLU) 的门控机制结合起来。我们还会省略线性层中有时使用的偏置项, 这遵循了自 PaLM [A. Chowdhery 等, 2022] 和 LLaMA [H. Touvron 等, 2023] 以来大多数现代大语言模型的做法。

SiLU 或 Swish 激活函数 [D. Hendrycks 等, 2016; S. Elfwing 等, 2017] 定义如下:

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (5)$$

如图 3 所示, SiLU 激活函数与 ReLU 类似, 但在零点处是光滑的。

门控线性单元 (GLU) 最初由 Y. N. Dauphin 等 [19] 定义: 一个线性变换经 sigmoid 函数处理后, 与另一个线性变换进行逐元素相乘:

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \quad (6)$$

其中, $\odot$ 表示逐元素乘法。关于门控线性单元, 原论文认为它能够 “在保留非线性能力的同时, 为梯度提供线性通路, 从而减轻深层架构中的梯度消失问题”。

将 SiLU / Swish 与 GLU 结合起来，就得到我们将在前馈网络中使用的 SwiGLU：

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2(\text{SiLU}(W_1 x) \odot W_3 x), \quad (7)$$

其中， $x \in \mathbb{R}^{d_{\text{model}}}$ ， $W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ ， $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ ，通常取 $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ 。具体实现时，可以将其舍入到附近的 64 的倍数，以提高硬件效率。

N. Shazeer [20] 最早提出将 SiLU / Swish 激活与 GLU 结合，并通过实验表明，在语言建模任务中，SwiGLU 优于 ReLU、SiLU（无门控）等基线。在作业后面，你将比较 SwiGLU 和 SiLU。尽管我们为这些组件提到了一些启发式解释，相关论文也提供了更多支持证据，但保持实证视角仍然很重要。Shazeer 论文中有一句如今已广为人知的话：

“我们不解释这些架构为何看起来有效；我们将它们的成功，如同其他一切一样，归功于神的仁慈。”

问题 (positionwise_feedforward)：实现逐位置前馈网络 (2 分)

提交内容：实现由 SiLU 激活函数与 GLU 组成的 SwiGLU 前馈网络。

注意：在这个特定情形下，为了数值稳定性，可以在实现中使用 `torch.sigmoid`。

实现时，应将 d_{ff} 设为大约 $\frac{8}{3} \times d_{\text{model}}$ ，同时确保前馈网络中间层的维度为 64 的倍数，以充分利用硬件。要用提供的测试检查实现，需要完成测试适配器 `adapters.run_swiglu`，然后运行 `uv run pytest -k test_swiglu`。

3.4.3 相对位置嵌入

为了向模型注入位置信息，我们将实现旋转位置嵌入 [J. Su 等, 2021]，通常称为 RoPE。对于词元位置 i 上的给定查询词元 $q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d$ ，应用一个成对旋转矩阵 R^i ，得到 $q'^{(i)} = R^i q^{(i)} = R^i W_q x^{(i)}$ 。这里， R^i 将嵌入元素对 $q_{2k-1:2k}^{(i)}$ 视作二维向量，按角度

$$\theta_{i,k} = \frac{i}{\Theta(2k-2)/d}$$

进行旋转，其中 $k \in \{1, \dots, d/2\}$ ， Θ 为某个常数。因此，可以将 R^i 看作一个大小为 $d \times d$ 的分块对角矩阵，其各个块为 R_k^i ， $k \in \{1, \dots, d/2\}$ ，且

$$R_k^i = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix} \quad (8)$$

于是，完整旋转矩阵为

$$R^i = \begin{pmatrix} R_1^i & 0 & 0 & \cdots & 0 \\ 0 & R_2^i & 0 & \cdots & 0 \\ 0 & 0 & R_3^i & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R_{d/2}^i \end{pmatrix}, \quad (9)$$

其中，0 表示 2×2 零矩阵。虽然可以构造完整的 $d \times d$ 矩阵，但好的实现应利用该矩阵的性质，更高效地完成变换。由于我们只关心给定序列内部词元间的相对旋转，可以跨层、跨不同批次复用计算得到的 $\cos(\theta_{i,k})$ 和 $\sin(\theta_{i,k})$ 。若要进一步优化，可以让所有层引用同一个 RoPE 模块，并在初始化时创建预先计算好的二维 sin 和 cos 缓冲区，通过 `self.register_buffer(persistent=False)` 注册，而不是使用 `nn.Parameter`，因为我们不希望学习这些固定的正弦和余弦值。对 $q^{(i)}$ 进行的相同旋转过程，也要应用于 $k^{(j)}$ ，使用对应的 R^j 旋转。注意，这一层没有可学习参数。

问题 (rope): 实现 RoPE (2 分)

提交内容: 实现 `RotaryPositionalEmbedding` 类，对输入张量应用 RoPE。

建议采用以下接口：

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)
```

构造 RoPE 模块，并按需创建缓冲区。

`theta: float`: RoPE 的 Θ 值。

`d_k: int`: 查询向量与键向量的维度。

`max_seq_len: int`: 输入的最大序列长度。

`device: torch.device | None = None`: 存储缓冲区的设备。

```
def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor
```

处理形状为 $(\dots, \text{seq_len}, d_k)$ 的输入张量，并返回相同形状的张量。注意，应支持具有任意数量批维度的 x 。假设词元位置是一个形状为 $(\dots, \text{seq_len})$ 的张量，用于指定 x 沿序列维度的词元位置。

应使用词元位置，沿序列维度切片你的 cos 和 sin 张量；这些张量可以是预先计算好的。

要测试实现，完成 `adapters.run_rope`，并确保通过 `uv run pytest -k test_rope`。

3.4.4 缩放点积注意力

现在，我们将实现 A. Vaswani 等 [8] 第 3.2.1 节描述的缩放点积注意力。作为准备，Attention 操作的定义需要使用 softmax；它将未归一化的分数向量转换为归一化分布：

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)}. \quad (10)$$

注意，当 v_i 很大时， $\exp(v_i)$ 可能变为 `inf`，进而产生 $\frac{\text{inf}}{\text{inf}} = \text{NaN}$ 。可以利用以下事实避免这一点：对所有输入同时加上任意常数 c ，`softmax` 的结果保持不变。借助这一性质可以提高数值稳定性——通常，从 v 的所有元素中减去其中的最大值，使新的最大元素为 0。现在，请使用这个数值稳定性技巧实现 `softmax`。

问题 (softmax)：实现 softmax (1 分)

提交内容：编写一个函数，对张量应用 `softmax`。函数应接收两个参数：一个张量和维度 i ；并沿输入张量的第 i 个维度应用 `softmax`。输出张量的形状应与输入相同，但第 i 个维度上的取值现在应构成归一化概率分布。请使用从第 i 个维度的所有元素中减去该维度最大值的技巧，避免数值稳定性问题。

要测试实现，完成 `adapters.run_softmax`，并确保通过：

```
uv run pytest -k test_softmax_matches_pytorch
```

现在可以从数学上定义 Attention 操作：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (11)$$

其中， $Q \in \mathbb{R}^{n \times d_k}$ 、 $K \in \mathbb{R}^{m \times d_k}$ 、 $V \in \mathbb{R}^{m \times d_v}$ 。这里， Q 、 K 、 V 都是该操作的输入，注意它们并不是可学习参数。**掩码：**有时需要对注意力操作的输出施加掩码。掩码应具有形状 $M \in \{\text{True}, \text{False}\}^{n \times m}$ ；这个布尔矩阵的第 i 行指明查询 i 应关注哪些键。按照通常的约定（这可能略有些令人困惑），位置 (i, j) 的值为 `True` 表示查询 i 会关注键 j ，值为 `False` 则表示查询不会关注该键。换言之，值为 `True` 的 (i, j) 位置允许“信息流动”。例如，对于一个 1×3 的掩码矩阵 `[[True, True, False]]`，唯一的查询向量仅关注前两个键。从计算角度看，使用掩码比在各个子序列上分别计算注意力高效得多。具体做法是取 `softmax` 之前的值 $QK^T/\sqrt{d_k}$ ，在掩码矩阵为 `False` 的所有对应位置加上 $-\infty$ 。

问题 (scaled_dot_product_attention)：实现缩放点积注意力 (5 分)

提交内容：实现缩放点积注意力函数。实现应支持形状为 `(batch_size, ..., seq_len, d_k)` 的键与查询，以及形状为 `(batch_size, ..., seq_len, d_v)` 的值；其中 `...` 表示任意数量的其他类批维度（如果提供）。返回的输出形状应为 `(batch_size, ..., seq_len, d_v)`。关于类批维度的讨论，见第 3.2 节。

实现还应支持用户可选提供的、形状为 `(seq_len, seq_len)` 的布尔掩码。掩码值为 `True` 的位置，其注意力概率总和应为 1；掩码值为 `False` 的位置，注意力概率应为 0。

要用我们提供的测试检查实现，请先完成以下测试适配器：

```
adapters.run_scaled_dot_product_attention
```

```
uv run pytest -k test_scaled_dot_product_attention
```

会使用三阶输入张量测试实现，而

```
uv run pytest -k test_4d_scaled_dot_product_attention
```

会使用四阶输入张量测试实现。

3.4.5 因果多头自注意力

我们将实现 A. Vaswani 等 [8] 第 3.2.2 节所述的多头自注意力。回顾其数学定义，多头注意力操作为：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \quad (12)$$

$$\text{其中 } \text{head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (13)$$

Q_i 、 K_i 、 V_i 分别是对 Q 、 K 、 V 的嵌入维度取出的第 $i \in \{1, \dots, h\}$ 个切片，大小为 d_k 或 d_v 。Attention 是第 3.4.4 节定义的缩放点积注意力操作。由此可以构造多头自注意力操作：

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x) \quad (14)$$

这里，可学习参数为 $W_Q \in \mathbb{R}^{hd_k \times d_{\text{model}}}$ 、 $W_K \in \mathbb{R}^{hd_k \times d_{\text{model}}}$ 、 $W_V \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 和 $W_O \in \mathbb{R}^{d_{\text{model}} \times hd_v}$ 。由于多头注意力操作会对 Q 、 K 和 V 进行切片，可以将 W_Q 、 W_K 和 W_V 看作沿输出维度为每个头分别划分。实现正确后，你应总共使用三次矩阵乘法计算键、值和查询投影。⁵

因果掩码

你的实现应阻止模型关注序列中的未来词元。换言之，如果给模型一个词元序列 $t_1, \dots, t_n$ ，希望计算前缀 $t_1, \dots, t_i$ （其中 $i < n$ ）对应的下一个词预测，那么模型不应能够访问或关注位置 $t_{i+1}, \dots, t_n$ 的词元表示。因为推理生成文本时，模型无法访问这些词元；而且这些未来词元会泄露真实下一个词的身份，使语言模型预训练目标变得轻而易举。对于输入序列 $t_1, \dots, t_n$ ，一种朴素的防止访问未来词元的方法，是对序列中 n 个不同前缀分别运行多头自注意力，共运行 n 次。我们将改用因果注意力掩码，让词元 i 能够关注序列中所有满足 $j \leq i$ 的位置。可以使用 `torch.triu` 或广播后的索引比较构造此掩码，并利用第 3.4.4 节的缩放点积注意力实现已经支持注意力掩码这一点。

应用 RoPE

RoPE 应用于查询向量和键向量，但不应用于值向量。另外，注意力头维度应作为批维度处理，因为多头注意力对每个头独立执行注意力运算。这意味着，每个头的查询向量和键向量都应应用完全相同的 RoPE 旋转。

问题 (multihead_self_attention)：实现因果多头自注意力 (5 分)

⁵ 作为进阶目标，可以尝试将键、查询和值投影合并到一个权重矩阵中，从而只需执行一次矩阵乘法。

提交内容：将因果多头自注意力实现为 `torch.nn.Module`。实现应至少接受以下参数：

`d_model: int`: Transformer 块输入的维度。

`num_heads: int`: 多头自注意力中使用的头数。

遵循 A. Vaswani 等 [8]，设 $d_k = d_v = \frac{d_{\text{model}}}{h}$ 。要用所提供的测试检查实现，完成测试适配器 `adapters.run_multihead_self_attention`，然后运行 `uv run pytest -k test_multihead_self_attention`。

3.5 完整的 Transformer 语言模型

先将 Transformer 块组装起来，回顾图 2 会有所帮助。一个 Transformer 块包含两个“子层”：一个用于多头自注意力，另一个用于 SwiGLU 前馈网络。每个子层都先执行 RMSNorm，然后执行主要操作（MHA / FF），最后加入残差连接。

具体而言，Transformer 块的前半部分，也就是第一个“子层”，应通过以下更新，从输入 x 产生输出 y ：

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \quad (15)$$

问题 (transformer_block)：实现 Transformer 块 (3 分)

按照第 3.4 节的描述和图 2 的示意，实现前置归一化 Transformer 块。你的 Transformer 块应至少接受以下参数：

`d_model: int`: Transformer 块输入的维度。

`num_heads: int`: 多头自注意力中使用的头数。

`d_ff: int`: 逐位置前馈网络中间层的维度。

要测试实现，完成适配器 `adapters.run_transformer_block`，然后运行 `uv run pytest -k test_transformer_block`。

提交内容：能够通过所提供测试的 Transformer 块代码。

现在，按照图 1 的高层次示意，将各个块组合起来。遵循第 3.1.0.1 节关于嵌入的描述，将嵌入送入 `num_layers` 个 Transformer 块，再经过最后的层归一化和 LM head，得到词表上的未归一化分布，即 logits。

问题 (transformer_lm)：实现 Transformer 语言模型 (3 分)

现在把一切组装起来！按照第 3.1 节的描述和图 1 的示意，实现 Transformer 语言模型。实现至少应接受前面 Transformer 块的全部构造参数，以及以下额外参数：

`vocab_size: int`: 词表大小，用于确定词元嵌入矩阵的维度。

`context_length: int`: 最大上下文长度, 用于确定 RoPE 的 $\sin$ 与 $\cos$ 缓冲区的维度。
`num_layers: int`: 使用的 Transformer 块数量。
要用所提供的测试检查实现, 首先完成测试适配器 `adapters.run_transformer_lm`, 然后运行 `uv run pytest -k test_transformer_lm`。
提交内容: 一个通过上述测试的 Transformer 语言模型模块。

资源核算

理解 Transformer 各部分如何消耗计算资源和内存很有用。下面将介绍基本“FLOPs 核算”的步骤。Transformer 中绝大多数浮点运算来自矩阵乘法, 因此核心方法很简单:

1. 写出 Transformer 一次前向传播中的所有矩阵乘法。
2. 将每个矩阵乘法换算为所需的 FLOPs。

第二步可以使用下面的事实: **规则**: 给定 $A \in \mathbb{R}^{m \times n}$ 和 $B \in \mathbb{R}^{n \times p}$, 矩阵乘积 AB 需要 $2mnp$ 次浮点运算。要理解这一点, 注意 $(AB)[i, j] = A[i, :] \cdot B[:, j]$, 这个点积需要 n 次加法和 n 次乘法, 即 $2n$ 次浮点运算。矩阵乘积 AB 有 $m \times p$ 个元素, 因此总 FLOPs 为 $(2n)(mp) = 2mnp$ 。

做下一题之前, 可以先逐个检查 Transformer 块和 Transformer 语言模型的各个组件, 列出所有矩阵乘法及其对应的 FLOPs 开销。

问题 (transformer_accounting): Transformer 语言模型资源核算 (5 分)

(a) 考虑一个采用本作业架构、规模与 GPT-2 XL 相当的模型, 配置如下:

`vocab_size`: 50,257
`context_length`: 1,024
`num_layers`: 48
`d_model`: 1,600
`num_heads`: 25
`d_ff`: 4,288 (最接近 $\frac{8}{3} \times 1,600$ 的 64 的倍数)

假设按该配置构造模型。模型有多少个可训练参数? 假设每个参数使用单精度浮点数表示, 仅加载该模型需要多少内存?

提交内容: 一至两句话回答。

(b) 找出这个 GPT-2 XL 形状模型完成一次前向传播所需的矩阵乘法。这些矩阵乘法总共需要多少 FLOPs? 假设输入序列包含 `context_length` 个词元。

提交内容：矩阵乘法列表（附说明），以及所需的总 FLOPs。

(c) 根据上面的分析，模型哪些部分需要最多 FLOPs？

提交内容：一至两句话回答。

(d) 对 GPT-2 small (12 层, d_{model} 为 768, 12 个头)、GPT-2 medium (24 层, d_{model} 为 1024, 16 个头) 和 GPT-2 large (36 层, d_{model} 为 1280, 20 个头) 重复分析。随着模型规模增加, Transformer 语言模型的哪些部分在总 FLOPs 中所占比例增加, 哪些部分所占比例减少？

提交内容：对每个模型给出各组件及其 FLOPs 的分解, 以一次前向传播总 FLOPs 的占比表示。此外, 用一至两句话说明, 改变模型规模如何影响各组件的 FLOPs 占比。

(e) 对 GPT-2 XL, 将上下文长度增加到 16,384。一次前向传播的总 FLOPs 如何变化？各模型组件对 FLOPs 的相对贡献如何变化？

提交内容：一至两句话回答。

4 训练 Transformer 语言模型

现在, 我们已经有了数据预处理步骤（分词器）和模型（Transformer）。接下来要构建支持训练的所有代码, 包括：

- **损失：**定义损失函数, 即交叉熵。
- **优化器：**定义用于最小化损失的优化器, 即 AdamW。
- **训练循环：**构建加载数据、保存检查点和管理训练所需的全部支持基础设施。

4.1 交叉熵损失

回顾一下, 对每个长度为 $m + 1$ 的序列 x 和 $i = 1, \dots, m$, Transformer 语言模型定义分布 $p_{\theta}(x_{i+1} | x_{1:i})$ 。给定由长度为 $m + 1$ 的序列组成的训练集 D , 标准的交叉熵（负对数似然）损失函数定义为：

$$\ell(\theta; D) = \frac{1}{|D|} \sum_{x \in D} \sum_{i=1}^m -\log p_{\theta}(x_{i+1} | x_{1:i}). \quad (16)$$

注意, Transformer 的一次前向传播即可得到所有 $i = 1, \dots, m$ 对应的 $p_{\theta}(x_{i+1} | x_{1:i})$ 。

具体而言, Transformer 在每个位置 i 计算 logits $o_i \in \mathbb{R}^{\text{vocab_size}}$, 于是有：⁶

$$p(x_{i+1} | x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])}. \quad (17)$$

交叉熵损失通常相对于 logits 向量 $o_i \in \mathbb{R}^{\text{vocab_size}}$ 和目标 x_{i+1} 来定义。⁷

⁶ 注意, $o_i[k]$ 表示向量 o_i 在索引 k 处的值。

⁷ 这对应于集中在 x_{i+1} 上的狄拉克 delta 分布与预测分布 $\text{softmax}(o_i)$ 之间的交叉熵。

与 softmax 一样，实现交叉熵损失时需要小心处理数值问题。

问题 (cross_entropy)：实现交叉熵 (1 分)

提交内容：编写一个计算交叉熵损失的函数，接收预测 logits (o_i) 和目标 (x_{i+1})，计算交叉熵 $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$ 。函数应处理以下事项：

- 减去最大元素，以保证数值稳定性。
- 尽可能将 log 和 exp 相互抵消。
- 处理任意额外批维度，并返回整个批的平均值。与第 3.2 节一样，假设类批维度总是在前，位于词表大小维度之前。

实现 `adapters.run_cross_entropy`，然后运行 `uv run pytest -k test_cross_entropy` 测试你的实现。

困惑度

训练时使用交叉熵就足够了，但在评估模型时，我们还希望报告困惑度。对于长度为 m 的序列，若各位置的交叉熵损失为 $\ell_1, \dots, \ell_m$ ，则：

$$\text{perplexity} = \exp \left(\frac{1}{m} \sum_{i=1}^m \ell_i \right). \quad (18)$$

4.2 SGD 优化器

有了损失函数之后，就可以开始探索优化器。最简单的基于梯度的优化器是随机梯度下降 (SGD)。从随机初始化的参数 θ_0 开始，在每一步 $t = 0, \dots, T - 1$ 执行以下更新：

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \quad (19)$$

其中， B_t 是从数据集 D 随机采样的一批数据，学习率 α_t 和批大小 $|B_t|$ 都是超参数。

4.2.1 在 PyTorch 中实现 SGD

实现优化器时，我们将继承 PyTorch 的 `torch.optim.Optimizer` 类。`Optimizer` 子类必须实现两个方法：

```
def __init__(self, params, ...)
```

该方法用于初始化优化器。这里，`params` 是待优化参数的集合，也可以是参数组，以便用户对模型不同部分使用不同超参数，例如不同学习率。务必将 `params` 传给基类的 `__init__` 方法，基类会保存这些参数，以供 `step` 使用。根据优化器的需要，你可以接收额外参数，常见例子是学习率；并以字典形式将它们传给基类构造函数，字典键是你为这些参数选取的名称字符串。

```
def step(self)
```

该方法用于执行一次参数更新。在训练循环中，它会在反向传播之后调用，因此能够访问上一批数据对应的梯度。该方法应遍历每个参数张量 `p`，并原地修改它们，即根据梯度 `p.grad`（如果存在），设置保存该参数对应张量的 `p.data`。

这里，`p.grad` 是表示损失相对于该参数的梯度的张量。

PyTorch 优化器 API 有一些细节，用示例解释会更直观。为了让例子更丰富，我们实现 SGD 的一个小变体：学习率随训练衰减，从初始学习率 α 开始，随着时间推移逐步减小步长：

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t) \quad (20)$$

下面看看如何将这个版本的 SGD 实现为 PyTorch 的 `Optimizer`：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"] # 获取学习率。
            for p in group["params"]:
                if p.grad is None:
                    continue
                state = self.state[p] # 获取与 p 关联的状态。
                t = state.get("t", 0) # 从状态中读取迭代次数；若不存在则取 0。
                grad = p.grad.data # 获取损失相对于 p 的梯度。
                p.data -= lr / math.sqrt(t + 1) * grad # 原地更新权重张量。
                state["t"] = t + 1 # 递增迭代次数。
        return loss
```

在 `__init__` 中，我们将参数以及默认超参数传递给基类构造函数；参数可能已经分组，每组有不同超参数。如果参数只是一个 `torch.nn.Parameter` 对象集合，基类构造函数会创建一个参数组，并为其分配默认超参数。接着，在 `step` 中，先遍历各参数组，再遍历组内各参数，并应用公式 20。这里，将迭代次数作为每个参数的关联状态保存：先读取这个值，在梯度更新中使用它，然后更新该状态值。API 规定，用户可能传入一个可调用的 `closure`，在优化器更新前重新计算损失。我们将使用的优化器不需要这项功能，但仍加入它以符合 API。

以下是一个最小训练循环示例，可用于观察上述优化器如何工作：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)
for t in range(100):
    opt.zero_grad() # 清空所有可学习参数的梯度。
    loss = (weights**2).mean() # 计算标量损失值。
    print(loss.cpu().item())
```

```
loss.backward() # 执行反向传播，计算梯度。  
opt.step() # 执行优化器更新。
```

这是训练循环的典型结构：每次迭代先计算损失，再执行一步优化器更新。训练语言模型时，可学习参数来自模型，在 PyTorch 中，`m.parameters()` 会返回这一集合。损失会在采样得到的一批数据上计算，但训练循环的基本结构保持不变。

问题 (learning_rate_tuning)：调整学习率 (1 分)

我们将看到，学习率是对训练影响最大的超参数之一。通过这个玩具示例实际观察一下：使用另外三个学习率值 $1e1$ 、 $1e2$ 和 $1e3$ 运行上面的 SGD 示例，每次只训练 10 次迭代。对于各个学习率，损失会发生什么变化？它下降得更快、更慢，还是发散，即在训练过程中增大？

提交内容：用一至两句话描述你观察到的现象。

4.3 AdamW

现代语言模型通常不用 SGD，而是采用更复杂的优化器进行训练。近来使用的大多数优化器都是 Adam 优化器 [D. P. Kingma 等, 2015] 的变体。我们将使用近期工作中广泛采用的 AdamW [I. Loshchilov 等, 2019]。AdamW 对 Adam 提出了一项修改：通过添加权重衰减来改善正则化，也就是每次迭代都将参数拉向 0，并让这一操作与梯度更新相解耦。我们将按照 I. Loshchilov 等 [23] 的算法 2 实现 AdamW。

AdamW 是有状态的：对每个参数，它都会跟踪一阶矩和二阶矩的运行估计。因此，AdamW 通过使用额外内存，换取更好的稳定性与收敛性。除学习率 α 外，AdamW 还有一对控制矩估计更新的超参数 (β_1, β_2) ，以及权重衰减率 λ 。典型应用将 (β_1, β_2) 设为 $(0.9, 0.999)$ ，但 LLaMA [H. Touvron 等, 2023] 和 GPT-3 [T. B. Brown 等, 2020] 等大语言模型经常使用 $(0.9, 0.95)$ 训练。该算法可以写成下面的形式，其中 ε 是一个小值，例如 10^{-8} ，用于在 v 极小时改善数值稳定性：

算法 1: AdamW 优化器

1	$\text{init}(\theta)$	▷ 初始化可学习参数
2	$m \leftarrow 0$	▷ 一阶矩向量初值; 形状与 θ 相同
3	$v \leftarrow 0$	▷ 二阶矩向量初值; 形状与 θ 相同
4	对于 $t = 1, \dots, T$ 执行	
5	采样一批数据 B_t	
6	$g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$	▷ 计算损失的梯度
7	$\alpha_t \leftarrow \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t}$	▷ 计算第 t 次迭代调整后的 α
8	$\theta \leftarrow \theta - \alpha \lambda \theta$	▷ 应用权重衰减
9	$m \leftarrow \beta_1 m + (1 - \beta_1)g$	▷ 更新一阶矩估计
10	$v \leftarrow \beta_2 v + (1 - \beta_2)g^2$	▷ 更新二阶矩估计
11	$\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v} + \varepsilon}$	▷ 应用按矩调整的权重更新
12	结束循环	

算法 1: AdamW 优化器伪代码。

注意, t 从 1 开始。现在请实现这个优化器。

问题 (adamw): 实现 AdamW (2 分)

提交内容: 将 AdamW 优化器实现为 `torch.optim.Optimizer` 的子类。在 `__init__` 中接收学习率 α , 以及 β 、 ε 和 λ 等超参数。为帮助维护状态, `Optimizer` 基类提供了字典 `self.state`, 它将 `nn.Parameter` 对象映射到一个字典, 用于存储该参数所需的任意信息; 对 AdamW 而言, 就是矩估计。实现 `adapters.get_adamw_cls`, 并确保通过 `uv run pytest -k test_adamw`。

问题 (adamw_accounting): 使用 AdamW 训练的资源核算 (2 分)

下面计算运行 AdamW 需要多少内存和计算量。假设所有张量都使用 `float32`。

(a) 运行 AdamW 所需的峰值内存是多少? 请按参数、激活值、梯度和优化器状态的内存用量分解答案。用 `batch_size` 和模型超参数 (`vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads`) 表示答案。假设 $d_{\text{ff}} = \frac{8}{3} \times d_{\text{model}}$

为简化起见, 计算激活值的内存用量时, 只考虑以下组件:

- Transformer 块
 - RMSNorm 层。
 - 多头自注意力子层: QKV 投影、 $QK^{\top}$ 矩阵乘法、softmax、值向量的加权和、输出投影。
 - 逐位置前馈网络 (SwiGLU): W_1 、 W_2 、门控分支上的 SiLU、逐元素乘积、 W_3 。

- 最后的 RMSNorm。
- 输出嵌入。
- 对 logits 计算交叉熵。

提交内容：分别给出参数、激活值、梯度和优化器状态的代数表达式，以及总量。

(b) 将答案代入 GPT-2 XL 形状的模型，得到一个仅依赖 `batch_size` 的表达式。在不超过 80 GB 内存的情况下，最大可用批大小是多少？

提交内容：形如 $a \cdot \text{batch_size} + b$ 的表达式，其中 a 、 b 为数值；另给出最大批大小。

(c) 运行一步 AdamW 需要多少 FLOPs？

提交内容：一个代数表达式，并简要说明理由。

(d) 模型 FLOPs 利用率 (MFU) 定义为实际观察到的吞吐量 (词元/秒) 相对于硬件理论峰值 FLOP 吞吐量的比值 [A. Chowdhery 等, 2022]。NVIDIA H100 GPU 的 “float32” 运算理论峰值为 495 teraFLOP/s；这里实际上指 TensorFloat-32，而它实际上是 “bfloat19”。假设可以达到 50% 的 MFU，在单张 H100 上，以批大小 1024 训练 GPT-2 XL 共 40 万步，需要多长时间？遵循 J. Kaplan 等 [25] 和 J. Hoffmann 等 [26]，假设反向传播的 FLOPs 是前向传播的两倍。

提交内容：训练所需的小时数，并简要说明理由。

4.4 学习率调度

使损失下降最快的学习率通常会随着训练而变化。训练 Transformer 时，通常采用学习率调度：先使用较大的学习率，在初期更快更新，然后随着训练逐渐衰减到较小的值。⁸ 本作业将实现用于训练 LLaMA 的余弦退火调度 [H. Touvron 等, 2023]。

调度器本质上是一个函数：输入当前步数 t 以及其他相关参数，例如初始和最终学习率，返回第 t 步梯度更新应使用的学习率。最简单的调度是常数函数，无论给定什么 t ，都返回同一个学习率。

余弦退火学习率调度接收以下参数：(i) 当前迭代 t ；(ii) 最大学习率 $\alpha_{\max}$ ；(iii) 最小（最终）学习率 $\alpha_{\min}$ ；(iv) 预热迭代次数 T_w ；(v) 余弦退火的最后一次迭代 T_c 。第 t 次迭代的学习率定义如下：（预热）若 $t < T_w$ ，则 $\alpha_t = \frac{t}{T_w} \alpha_{\max}$ 。

（余弦退火）若 $T_w \leq t \leq T_c$ ，则

$$\alpha_t = \alpha_{\min} + \frac{1}{2} \left(1 + \cos \left(\frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{\max} - \alpha_{\min}).$$

（退火后）若 $t > T_c$ ，则 $\alpha_t = \alpha_{\min}$ 。

问题 (learning_rate_schedule)：实现带预热的余弦学习率调度 (1 分)

⁸ 有时也常使用让学习率重新升高，即“重启”的调度，以帮助越过局部极小值。

编写一个函数，接收 t 、 $\alpha_{\max}$ 、 $\alpha_{\min}$ 、 T_w 和 T_c ，并根据上述调度器返回学习率 α_t 。然后实现 `adapters.get_lr_cosine_schedule`，确保通过 `uv run pytest -k test_get_lr_cosine_schedule`。

4.5 梯度裁剪

训练中有时会遇到产生很大梯度的训练样本，导致训练不稳定。为缓解这一问题，实践中经常使用梯度裁剪：在每次反向传播之后、优化器更新之前，对梯度范数施加上限。

给定全部参数的梯度 g ，计算其 ℓ_2 范数 $\|g\|_2$ 。若该范数小于最大值 M ，就保持 g 不变；否则，将 g 乘以缩放因子

$$\frac{M}{\|g\|_2 + \varepsilon},$$

其中加入一个很小的 ε ，例如 10^{-6} ，以保证数值稳定性。注意，缩放后的范数会略小于 M 。

问题 (gradient_clipping)：实现梯度裁剪 (1 分)

编写梯度裁剪函数。函数应接收参数列表和最大 ℓ_2 范数，并原地修改各参数的梯度。使用 $\varepsilon = 10^{-6}$ ，即 PyTorch 默认值。然后实现适配器 `adapters.run_gradient_clipping`，确保通过 `uv run pytest -k test_gradient_clipping`。

5 训练循环

现在，终于可以把之前构建的主要组件组合起来：分词后的数据、模型和优化器。

5.1 数据加载器

分词后的数据，例如你在 `tokenizer_experiments` 中准备的数据，是一个单独的词元序列 $x = (x_1, \dots, x_n)$ 。即使原始数据由不同文档组成，例如不同网页或源代码文件，常见做法仍是将它们全部拼接为一个词元序列，并在文档之间添加分隔符，例如 `<|endoftext|>` 词元。

数据加载器将该序列转换为一连串批次。每批包含 B 个长度为 m 的序列，并与同样长为 m 的对应下一个词元目标配对。例如，当 $B = 1$ 、 $m = 3$ 时， $([x_2, x_3, x_4], [x_3, x_4, x_5])$ 就是一个可能的批次。

这种数据加载方式从多个方面简化了训练。首先，任意 $1 \leq i \leq n - m$ 都对应一个有效的训练序列，因此采样训练序列非常简单。由于所有训练序列等长，不必填充输入序列，这会提高硬件利用率，也可以通过增大批大小 B 进一步改善。最后，采样训练数据时不需要加载完整数据集，因此容易处理那些原本可能无法放入内存的大型数据集。

问题 (data_loading)：实现数据加载 (2 分)

提交内容：编写一个函数，接收 NumPy 数组 x （包含词元 ID 的整数数组）、`batch_size`、`context_length` 和 PyTorch 设备字符串，例如 `'cpu'` 或 `'cuda:0'`，返回一对张量：采样得到的输入序列，以及对应的下一个词元目标。这两个

张量都应具有形状 (batch_size, context_length), 包含词元 ID, 并放在指定设备上。要用我们提供的测试检查实现, 首先完成测试适配器 `adapters.run_get_batch`, 然后运行 `uv run pytest -k test_get_batch`。

低资源提示: 在 CPU 或 Apple Silicon 上加载数据

如果计划在 CPU 或 Apple Silicon 上训练语言模型, 需要将数据移动到正确设备; 类似地, 稍后也应将模型放在同一个设备上。

如果使用 CPU, 可以使用设备字符串 'cpu'; 如果使用 Apple Silicon (M 系列芯片), 可以使用 'mps'。

关于 MPS 的更多信息, 请参阅:

- <https://docs.pytorch.org/docs/stable/mps.html>
- <https://docs.pytorch.org/docs/stable/notes/mps.html>
- <https://developer.apple.com/documentation/metalperformanceshaders>

如果数据集过大, 无法加载到内存中怎么办? 可以使用名为 `mmap` 的 Unix 系统调用, 将磁盘文件映射到虚拟内存, 并在访问相应内存位置时惰性加载文件内容。这样, 就可以“假装”整个数据集都在内存中。NumPy 通过 `np.memmap` 实现这一功能; 如果最初使用 `np.save` 保存数组, 也可以给 `np.load` 传入 `mmap_mode='r'`。这会返回一个类似 NumPy 数组的对象, 在访问元素时按需加载。**训练时从数据集, 也就是 NumPy 数组中采样, 务必以内存映射模式加载数据集:** 根据数组的保存方式, 使用 `np.memmap` 或给 `np.load` 传入 `mmap_mode='r'`。还应确保指定的 `dtype` 与要加载的数组匹配。显式检查内存映射数据是否正确可能很有帮助, 例如检查其中是否包含超出预期词表大小的值。

5.2 检查点保存

除加载数据外, 训练过程中还需要保存模型。运行作业时, 我们经常希望恢复中途停止的训练, 例如作业超时或机器故障导致的中断。即使一切顺利, 也可能希望日后访问中间阶段的模型, 例如事后研究训练动态, 或从不同训练阶段的模型中采样。

检查点应包含恢复训练所需的全部状态。至少要能够恢复模型权重。如果使用 AdamW 等有状态优化器, 还需要保存优化器状态; 对 AdamW 而言, 就是矩估计。最后, 为恢复学习率调度, 还需要知道停止时的迭代次数。PyTorch 使这些状态的保存非常容易: 每个 `nn.Module` 都有 `state_dict()` 方法, 返回包含全部可学习权重的字典; 之后可以用对应的 `load_state_dict()` 恢复权重。任何 `torch.optim.Optimizer` 也同样如此。最后, `torch.save(obj, dest)` 可以将对象写入文件路径或类文件对象; 该对象例如可以是一个字典, 既包含张量值, 也包含整数等普通 Python 对象。之后使用 `torch.load(src)` 将其重新加载到内存。

问题 (checkpointing): 实现模型检查点保存 (1 分)

实现以下两个函数，用于加载与保存检查点：

```
def save_checkpoint(model, optimizer, iteration, out)
```

该函数应将 `model`、`optimizer` 和 `iteration` 的全部状态写入类文件对象 `out`。可以使用模型和优化器的 `state_dict` 方法获取相关状态，再通过 `torch.save(obj, out)` 将 `obj` 写入 `out`；PyTorch 在这里既支持路径，也支持类文件对象。通常可将 `obj` 设为字典，不过只要之后能够加载检查点，就可以使用任意格式。

此函数应接受以下参数：

```
model: torch.nn.Module
optimizer: torch.optim.Optimizer
iteration: int
out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
```

```
def load_checkpoint(src, model, optimizer)
```

该函数应从 `src`（路径或类文件对象）加载检查点，并从中恢复模型与优化器状态。函数应返回检查点中保存的迭代次数。可通过 `torch.load(src)` 恢复 `save_checkpoint` 实现中保存的内容，再调用模型和优化器各自的 `load_state_dict` 方法，将它们恢复到原来的状态。

此函数应接受以下参数：

```
src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
model: torch.nn.Module
optimizer: torch.optim.Optimizer
```

实现 `adapters.run_save_checkpoint` 和 `adapters.run_load_checkpoint` 这两个适配器，并确保通过 `uv run pytest -k test_checkpointing`。

5.3 训练循环

现在终于可以在主训练脚本中，将所有已实现的组件组合起来。让脚本能够方便地以不同超参数启动训练非常值得，例如通过命令行参数传入超参数；因为后面会多次启动训练，研究不同选择如何影响训练过程。

问题 (training_together): 将各部分整合起来 (4 分)

提交内容：编写一个脚本，运行训练循环，在用户提供的输入上训练模型。具体而言，建议训练脚本至少支持：

- 配置与控制各种模型和优化器超参数。
- 使用 `np.memmap` 以内存高效的方式加载大型训练集和验证集。
- 将检查点序列化保存到用户提供的路径。

- 定期记录训练和验证性能，例如输出到控制台，和/或记录到 Weights and Biases 等外部服务。⁹

6 生成文本

现在已经能够训练模型，最后还需要让模型生成文本。回顾一下，语言模型接收一个长度为 `sequence_length` 的整数序列，也可以是批量序列，并生成一个大小为 `(sequence_length, vocab_size)` 的矩阵，序列中的每一项都是预测该位置之后下一个词元的概率分布。下面将编写一些函数，将其转换为新序列的采样方案。

Softmax

按照标准约定，语言模型输出的是最后一个线性层的输出，即“logits”，因此还必须通过 softmax 将它转换为归一化概率；该操作已在公式 10 中介绍。

解码

要从模型生成文本，也就是解码，我们先向模型提供前缀词元序列（“提示词”，prompt），让模型输出词表上的概率分布，用于预测序列中的下一个词元。然后，从这个词表项分布中采样，确定下一个输出词元。

具体而言，解码过程的一步应接收序列 $x_{1\dots t}$ ，并通过以下公式返回词元 x_{t+1} ：

$$P(x_{t+1} = i \mid x_{1\dots t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)} \quad (21)$$

$$v = \text{TransformerLM}(x_{1\dots t})_t \in \mathbb{R}^{\text{vocab_size}} \quad (22)$$

其中，TransformerLM 是我们的模型，接收长度为 `sequence_length` 的序列，并产生大小为 `(sequence_length, vocab_size)` 的矩阵。由于我们要获得第 t 个位置对应的下一个词元预测，因此取该矩阵的最后一项。

不断从这些单步条件分布中采样，将之前生成的输出词元追加到下一个解码时间步的输入，直到生成序列结束词元 `<|endoftext|>`，或达到用户指定的最大生成词元数，就构成了一个基本解码器。

解码技巧

我们将使用小模型做实验，而小模型有时会生成质量很差的文本。两个简单的解码技巧有助于改善这些问题。首先是**温度缩放**：通过温度参数 τ 修改 softmax，得到：

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\text{vocab_size}} \exp(v_j/\tau)}. \quad (23)$$

注意，当 $\tau \rightarrow 0$ 时， v 中的最大元素会占据主导地位，softmax 输出趋于一个集中在该最大元素处的 one-hot 向量。

⁹<https://wandb.ai>

其次，另一种技巧是**核采样 (nucleus sampling) 或 top-p 采样**：通过截去低概率词元来修改采样分布。令 q 为从经过温度缩放的 softmax 得到的、大小为 `vocab_size` 的概率分布。超参数为 p 的核采样按下式产生下一个词元：

$$P(x_{t+1} = i \mid q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j}, & \text{若 } i \in V(p), \\ 0, & \text{其他情况。} \end{cases} \quad (24)$$

其中， $V(p)$ 是使 $\sum_{j \in V(p)} q_j \geq p$ 成立的最小索引集合。可以先按概率大小排序 q ，再依次选择概率最大的词表项，直到累计概率达到目标水平 p ，从而轻松计算该集合。

问题 (decoding)：解码 (3 分)

提交内容：实现一个从语言模型中解码的函数。建议支持以下功能：

- 为用户提供的提示词生成续写，即接收某个 $x_{1\dots t}$ ，采样续写直到遇到 `<|endoftext|>` 词元。
- 允许用户控制生成词元数量的上限。
- 给定所需温度值，在采样前对预测的下一个词元分布应用 softmax 温度缩放。
- 给定用户指定的阈值，执行 top- p 采样 [A. Holtzman 等，2020]，也称核采样。

7 实验

现在，可以将一切组合起来，在预训练数据集上训练（小型）语言模型。

7.1 如何开展实验及提交要求

理解 Transformer 各架构组件设计依据的最佳方式，是亲自修改并运行模型。动手实践无可替代。

为此，你需要能够**快速、一致地开展实验，并记录做过什么**。为了快速实验，我们将在小规模模型（总参数量约 1700 万）和简单数据集（TinyStories）上运行许多实验。为保持一致性，需要系统地消融组件、改变超参数；为保留记录，我们会要求提交实验日志以及每次实验对应的学习曲线。

为了能够提交损失曲线，**请务必定期评估验证损失，并同时记录步数和实际经过时间 (wall-clock time)**。Weights and Biases 等日志基础设施可能会有所帮助。

问题 (experiment_log)：实验日志 (3 分)

为训练与评估代码建立实验跟踪基础设施，以便跟踪各项实验，以及损失曲线随梯度更新步数和实际经过时间的变化。

提交内容：实验日志基础设施代码，以及记录本节下列作业问题中所有尝试的实验日志文档。

7.2 TinyStories

我们将从一个非常简单的数据集开始：TinyStories (R. Eldan 等 [1])。模型在这个数据集上训练很快，也能观察到一些有趣的行为。获取数据集的说明见第 1 节，下面是一个数据示例。

示例 (tinystories_example)：TinyStories 中的一个例子

从前有个小男孩，名叫本。本喜欢探索身边的世界。他见过许多令人惊叹的东西，比如商店里陈列的漂亮花瓶。一天，本在商店里走着，发现了一个非常特别的花瓶。看到它，本惊讶极了！他说：“哇，这真是一个了不起的花瓶！我能买下它吗？”店主微笑着说：“当然可以。你可以把它带回家，让所有朋友看看它有多棒！”于是，本把花瓶带回家，感到无比自豪！他把朋友们叫来，向他们展示这个令人惊叹的花瓶。所有朋友都觉得花瓶很漂亮，简直不敢相信本竟然这么幸运。这就是本在商店里发现一个奇妙花瓶的故事！

7.2.1 超参数调优

我们会提供一些最基本的起始超参数，并让你为其他超参数寻找有效设置。**词表大小：10000**。典型词表大小在数万至数十万之间。你应改变这一设置，观察词表和模型行为如何变化。**上下文长度：256**。TinyStories 这样的简单数据集可能不需要很长的序列，但对后面使用的 OpenWebText 数据，你可能需要改变这一设置。尝试不同上下文长度，观察对每次迭代运行时间和最终困惑度的影响。**d_model：512**。这比许多小型 Transformer 论文使用的 768 维略小，但可以让训练更快。**d_ff：1344**。这大约是 $\frac{8}{3}d_{\text{model}}$ ，同时是 64 的倍数，有利于 GPU 性能。**RoPE 的 theta 参数 Θ ：10000**。**层数与头数：4 层、16 个头**。这些配置合起来约有 1700 万个非嵌入参数，是一个相当小的 Transformer。**处理的词元总数：327,680,000**。批大小 $\times$ 总步数 $\times$ 上下文长度应大致等于这个值。对于其余超参数，应通过一些试错找到良好的默认值：学习率、学习率预热、其他 AdamW 超参数 (β_1 、 β_2 、 ε) 以及权重衰减。这些超参数的典型选择可参考 D. P. Kingma 等 [22]。

7.2.2 将各部分整合起来

现在可以准备训练好的 BPE 分词器，对训练数据集进行分词，再使用自己编写的训练循环运行训练，从而将一切组合起来。**重要说明：**如果实现正确且高效，使用上述超参数在 1 张 B200

GPU 上的运行时间应约为 20 至 30 分钟。如果运行时间明显更长，请检查数据加载、检查点保存或验证损失代码是否成为瓶颈，并确认实现进行了正确的批处理。

7.2.3 模型架构调试技巧

我们强烈建议熟悉 IDE 内置的调试器，例如 VSCode / Zed 的调试器；与依赖 print 语句调试相比，这会节省时间。如果使用文本编辑器，可以使用 ipdb 等工具。调试模型架构时，其他一些良好实践包括：

- 开发任何神经网络架构时，一个常见的第一步是让模型过拟合单个小批次。如果实现正确，应该能够很快将训练损失降至接近零。
- 在模型各组件中设置调试断点，检查中间张量的形状，确保符合预期。
- 监控激活值、模型权重与梯度的范数，确保没有爆炸或消失。

问题 (learning_rate)：调整学习率 (2 B200 小时) (3 分)

学习率是最重要的待调超参数之一。使用已经训练的基础模型，回答以下问题：

(a) 对学习率进行超参数扫描，报告最终损失；如果优化器发散，请注明。

提交内容：不同学习率对应的学习曲线，并说明超参数搜索策略。

提交内容：一个在 TinyStories 上逐词元验证损失不超过 1.45 的模型。

低资源提示：在 CPU 或 Apple Silicon 上训练少量步数

如果在 cpu 或 mps 上运行，应将处理的词元总数降至 40,000,000，这已足以生成相当流畅的文本。目标验证损失也可以从 1.45 放宽到 2.00。

在配备 36 GB 内存的 M4 Max 芯片上运行我们的参考代码，并使用调优后的学习率，设置批大小 $\times$ 总步数 $\times$ 上下文长度 $= 32 \times 5000 \times 256 = 40,960,000$ 个词元，使用 cpu 耗时 1 小时 22 分钟，使用 mps 耗时 36 分钟。第 5000 步时，验证损失为 1.80。

其他一些提示：

- 使用 N 个训练步时，建议调整余弦学习率衰减调度，使其恰好在第 N 步结束衰减，即达到最小学习率。
- 使用 mps 时，不要使用 TF32 内核，即不要像在 cuda 设备上可能做的那样设置：

```
torch.set_float32_matmul_precision('high')
```

我们尝试在 mps 上启用 TF32 内核 (torch 版本 2.9.0)，发现后端有时会使用存在隐蔽错误的内核，导致训练不稳定。

- 可以用 `torch.compile` 对模型进行 JIT 编译，以加快训练。具体而言：

- 在 `cpu` 上，使用以下代码编译模型：

```
model = torch.compile(model)
```

- 在 `mps` 上，可以使用以下代码，对反向传播进行一定优化：

```
model = torch.compile(model, backend="aot_eager")
```

截至 `torch 2.9.0`，`mps` 尚不支持使用 `Inductor` 编译。

(b) 一种经验说法是，最好的学习率位于“稳定性的边缘”。研究学习率开始导致发散的临界点与你找到的最佳学习率之间有什么关系。

提交内容：一组逐渐增大学习率的学习曲线，其中至少包含一次发散的运行，并分析这与收敛速度的关系。

现在，改变批大小，看看训练会发生什么。批大小很重要：通过执行更大的矩阵乘法，可以提高 GPU 的使用效率。但是否总应该使用大批大小呢？让我们通过实验寻找答案。

问题 (batch_size_experiment)：改变批大小 (1 B200 小时) (1 分)

将批大小从 1 一直增大到 GPU 显存允许的上限。中间至少尝试若干个批大小，包括 64 和 128 等典型值。

提交内容：不同批大小的训练运行对应的学习曲线。必要时，应重新优化学习率。

提交内容：用几句话讨论你对批大小及其训练影响的发现。

有了解码器，现在可以生成文本了！我们将从模型中生成文本，看看效果如何。作为参考，你得到的输出质量至少应与下面的示例相当。

示例 (ts_generate_example)：TinyStories 语言模型的输出样本

从前，有个漂亮的小女孩，名叫莉莉。她喜欢吃口香糖，尤其是那块又大又黑的。一天，莉莉的妈妈请她帮忙做晚饭。莉莉兴奋极了！她喜欢帮助妈妈。莉莉的妈妈做了一大锅汤作为晚饭。莉莉很高兴，说：“谢谢你，妈妈！我爱你。”她帮妈妈把汤倒进一个大碗里。晚饭后，莉莉的妈妈做了一些美味的汤。莉莉很喜欢！她说：“谢谢你，妈妈！这汤真好喝！”妈妈微笑着说：“我很高兴你喜欢它，莉莉。”她们做完饭后，又继续一起做饭。故事结束。

低资源提示：在 CPU 或 Apple Silicon 上生成文本

如果采用处理 4000 万个词元的低资源配置，生成结果仍应看起来像英语，但不会像上面那样流畅。例如，下面是我们在 4000 万个词元上训练的 TinyStories 语言模型的一段输出：

从前，有个小女孩，名叫苏。苏有一颗她非常喜欢的牙齿。那是他最好的头。一天，苏出去散步，遇到了一只瓢虫！她们成了好朋友，一起在小路上玩耍。

“嘿，波莉！我们出去吧！”蒂姆说。苏看着天空，发现很难找到一种跳舞发光的方法。她微笑着，同意帮助说话！“

当苏看着天空移动，是什么。她

下面是具体题目及要求：

问题 (generate)：生成文本 (1 分)

使用你的解码器和训练好的检查点，报告模型生成的文本。为了获得流畅输出，可能需要调整解码器参数，例如温度、top-p 等。

提交内容：至少 256 个词元的生成文本，或直到第一个 `<|endoftext|>` 词元出现为止；另简要评价输出的流畅性，并讨论至少两个影响输出好坏的因素。

7.3 消融实验与架构修改

理解 Transformer 的最佳方式，是亲自修改它并观察其行为。下面将做几个简单的消融实验与修改。

消融 1：层归一化

人们常说，层归一化对 Transformer 训练的稳定性很重要。但也许我们想冒点险。把每个 Transformer 块中的 RMSNorm 移除，看看会发生什么。

问题 (layer_norm_ablation)：移除 RMSNorm 并训练 (0.5 B200 小时) (1 分)

移除 Transformer 中的全部 RMSNorm，再进行训练。使用之前的最佳学习率会发生什么？使用更低的学习率能否恢复稳定性？

提交内容：移除 RMSNorm 后训练的学习曲线，以及最佳学习率对应的学习曲线。

提交内容：用几句话评论 RMSNorm 的影响。

接下来研究另一个乍看之下似乎有些随意的层归一化选择。前置归一化 Transformer 块定义为：

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)) \quad (25)$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)). \quad (26)$$

这是对原始 Transformer 架构少数形成“共识”的修改之一。原始架构采用后置归一化方式：

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)) \quad (27)$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)). \quad (28)$$

现在恢复后置归一化，看看会发生什么。

问题 (pre_norm_ablation)：实现后置归一化并训练 (0.5 B200 小时) (1 分)

将前置归一化 Transformer 实现修改为后置归一化版本。训练后置归一化模型，观察结果。

提交内容：后置归一化 Transformer 的学习曲线，并与前置归一化版本比较。

我们看到，层归一化对 Transformer 的行为有重大影响，甚至归一化的位置也很重要。

消融 2：位置嵌入

接下来研究位置嵌入对模型性能的影响。具体而言，将比较基础模型（使用 RoPE）与完全不包含位置嵌入的模型（NoPE）。事实表明，仅解码器 Transformer，也就是像我们的实现一样具有因果掩码的 Transformer，即使没有显式提供位置嵌入，理论上也可以推断相对或绝对位置信息 [Y.-H. H. Tsai 等, 2019; A. Kazemnejad 等, 2023]。现在通过实验检验 NoPE 与 RoPE 的表现差异。

问题 (no_pos_emb)：实现 NoPE (0.5 B200 小时) (1 分)

修改使用 RoPE 的 Transformer 实现，完全移除位置嵌入信息，观察结果。

提交内容：比较 RoPE 与 NoPE 性能的学习曲线。

消融 3：SwiGLU 与 SiLU

接下来，遵循 N. Shazeer [20]，通过比较 SwiGLU 前馈网络与使用 SiLU 激活但不包含门控线性单元（GLU）的前馈网络，检验前馈网络中门控的重要性：

$$\text{FFN}_{\text{SiLU}}(x) = W_2 \text{SiLU}(W_1 x). \quad (29)$$

回顾一下，我们的 SwiGLU 实现将前馈网络中间层维度设为大约 $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ ，并确保 $d_{\text{ff}} \bmod 64 = 0$ ，以利用 GPU Tensor Core。在本消融基线中， FFN_{SiLU} 实现应改用 $d_{\text{ff}} = 4 \times d_{\text{model}}$ ，从而使参数量与默认 SwiGLU 前馈网络大致相同；后者有三个权重矩阵，而不是两个。

问题 (swiglu_ablation)：SwiGLU 与 SiLU (0.5 B200 小时) (1 分)

提交内容：在参数量大致匹配的情况下，比较 SwiGLU 和 SiLU 前馈网络性能的学习曲线。

提交内容：用几句话讨论你的发现。

低资源提示：GPU 资源有限的线上学生应在 TinyStories 上测试修改

在作业的剩余部分，我们将转向一个规模更大、噪声更多的网页数据集 OpenWebText，实验各种架构修改，并可选择向课程排行榜提交结果。

在 OpenWebText 上训练语言模型直至能够流畅生成文本，需要很长时间。因此，我们建议 GPU 使用机会有限的线上学生继续在 TinyStories 上测试修改，并以验证损失作为性能评估指标。

7.4 在 OpenWebText 上运行

现在转向一种更标准的、来自网页抓取的预训练数据集。我们同样提供了 OpenWebText [A. Gokaslan 等，2019] 的一小部分样本，保存为单个文本文件；获取方式见第 1 节。

下面是 OpenWebText 的一个例子。注意，它的文本更加贴近现实、复杂且多样。你可以浏览训练数据集，了解网页抓取语料中的训练数据是什么样的。

示例 (owt_example)：OWT 中的一个例子

Baseball Prospectus 的技术总监 Harry Pavlidis 在聘用 Jonathan Judge 时，冒了一次险。Pavlidis 知道，正如 Alan Schwarz 在《数字游戏》(The Numbers Game) 中所写：“在美国文化的任何角落，都没有什么比棒球运动员的表现更被精确地统计、更被热情地量化。”只需点几下鼠标，就可以知道 Noah Syndergaard 的快速球飞向本垒时，每分钟旋转超过 2,100 次；Nelson Cruz 在 2016 年的合资格击球手中拥有最高的平均击球初速；还有无数其他小知识，仿佛出自电子游戏或科幻小说。不断上涨的数据海洋，让棒球文化中一种日益重要的角色获得了力量：业余分析爱好者。

这种赋能也带来了更严格的审视——既针对测量结果，也针对背后的人员和出版机构。身处 Baseball Prospectus，Pavlidis 深知量化结果不够完善时会遭遇怎样的反弹。他也知道，网站的捕手指标需要重新设计，而完成这项工作需要一位学识深厚、能够处理复杂统计建模问题的人。“他把我们吓住了。”——Harry Pavlidis

根据 Judge 的文章，以及两人在网站赞助的一次球场活动中的交流，Pavlidis 直觉上认为 Judge “懂门道”。[……]

注意：本实验可能需要重新调整学习率、批大小等超参数。

问题 (main_experiment)：在 OWT 上进行实验 (2 B200 小时) (2 分)

采用与 TinyStories 实验相同的模型架构和总训练迭代次数，在 OpenWebText 上训练语言模型。这个模型表现如何？

提交内容：语言模型在 OpenWebText 上的学习曲线。说明损失与 TinyStories 有什么不同，以及应如何解释这些损失。

提交内容：OpenWebText 语言模型生成的文本，格式与 TinyStories 输出相同。文本流畅性如何？为什么模型和计算预算都与 TinyStories 相同，输出质量却更差？

7.5 你自己的修改与排行榜

恭喜你完成到这里，马上就要结束了！现在尝试改进 Transformer 架构，看看你的超参数和架构相较于班上其他同学表现如何。

排行榜规则

除以下规则外，没有其他限制：**运行时间：**提交内容在一张 B200 上最多运行 45 分钟。如果使用 SLURM 或 Modal，可能需要在提交脚本中强制执行该限制。**数据：**只能使用我们提供的 OpenWebText 训练数据集。除此之外，你可以自由尝试任何想法。

如果想找一些实现思路，可以查看以下资源：

- 最先进的开源大语言模型家族，例如 Llama 3 [A. Grattafiori 等，2024] 和 Qwen 2.5 [A. Yang 等，2024]。
- NanoGPT 速通仓库：<https://github.com/KellerJordan/modded-nanogpt>。社区成员在这里发布许多有趣修改，用于“速通”小规模语言模型预训练。例如，一项可追溯至原始 Transformer 论文的常见修改，是将输入嵌入和输出嵌入的权重绑定在一起，参见 A. Vaswani 等 [8] 第 3.4 节和 A. Chowdhery 等 [16] 第 2 节。如果尝试权重绑定，可能需要减小嵌入 / LM head 初始化的标准差。

在尝试完整的 45 分钟运行之前，应先在 OpenWebText 的一个小子集或 TinyStories 上测试这些修改。

需要提醒的是，一些在本排行榜上效果良好的修改，不一定能够推广到更大规模的预训练。我们将在课程的缩放定律单元进一步探讨这一点。

问题 (leaderboard)：排行榜 (10 B200 小时) (6 分)

按照上述排行榜规则训练一个模型，目标是在 0.75 B200 小时内，尽可能降低语言模型的验证损失。

提交内容：记录到的最终验证损失、一条明确使用实际经过时间作为横轴且显示运行时间小于 45 分钟的学习曲线，以及对所做修改的说明。我们期望排行榜提交至少优于损失为 5.0 的朴素基线。请在此提交到排行榜：<https://github.com/stanford-cs336/assignment1-basics-leaderboard>

参考文献

- [1] R. Eldan 和 Y. Li, 《TinyStories: 语言模型可以小到什么程度，仍能说出连贯的英语?》，2023。
- [2] A. Gokaslan、V. Cohen、E. Pavlick 和 S. Tellex, 《OpenWebText 语料库》，2019。
- [3] R. Sennrich、B. Haddow 和 A. Birch, 《使用子词单元进行罕见词的神经机器翻译》，载于 ACL 会议论文集, 2016。
- [4] C. Wang、K. Cho 和 J. Gu, 《使用字节级子词的神经机器翻译》，2019。
- [5] P. Gage, 《一种新的数据压缩算法》，C Users Journal, 第 12 卷, 第 2 期, 第 23–38 页, 1994 年 2 月。
- [6] A. Radford、J. Wu、R. Child、D. Luan、D. Amodei 和 I. Sutskever, 《语言模型是无监督多任务学习器》，2019。
- [7] A. Radford、K. Narasimhan、T. Salimans 和 I. Sutskever, 《通过生成式预训练改进语言理解》，2018。
- [8] A. Vaswani 等, 《注意力就是你所需要的一切》，载于 NeurIPS 会议论文集, 2017。
- [9] T. Q. Nguyen 和 J. Salazar, 《不再令人头疼的 Transformer: 改进自注意力的归一化》，载于 IWSWLT 会议论文集, 2019。
- [10] R. Xiong 等, 《论 Transformer 架构中的层归一化》，载于 ICML 会议论文集, 2020。
- [11] J. L. Ba、J. R. Kiros 和 G. E. Hinton, 《层归一化》，2016。
- [12] H. Touvron 等, 《LLaMA: 开放且高效的基础语言模型》，2023。
- [13] B. Zhang 和 R. Sennrich, 《均方根层归一化》，载于 NeurIPS 会议论文集, 2019。
- [14] A. Grattafiori 等, 《Llama 3 模型家族》。[在线] 可访问: <https://arxiv.org/abs/2407.21783>
- [15] A. Yang 等, 《Qwen2.5 技术报告》，arXiv 预印本 arXiv:2412.15115, 2024。
- [16] A. Chowdhery 等, 《PaLM: 借助 Pathways 扩展语言建模规模》，2022。
- [17] D. Hendrycks 和 K. Gimpel, 《通过高斯误差线性单元连接非线性与随机正则化》，2016。
- [18] S. Elfwing、E. Uchibe 和 K. Doya, 《用于强化学习中神经网络函数近似的 Sigmoid 加权线性单元》。[在线] 可访问: <https://arxiv.org/abs/1702.03118>
- [19] Y. N. Dauphin、A. Fan、M. Auli 和 D. Grangier, 《使用门控卷积网络进行语言建模》。[在线] 可访问: <https://arxiv.org/abs/1612.08083>
- [20] N. Shazeer, 《GLU 变体改进 Transformer》，2020。

- [21] J. Su、Y. Lu、S. Pan、B. Wen 和 Y. Liu, 《RoFormer: 使用旋转位置嵌入增强 Transformer》, 2021。
- [22] D. P. Kingma 和 J. Ba, 《Adam: 一种随机优化方法》, 载于 ICLR 会议论文集, 2015。
- [23] I. Loshchilov 和 F. Hutter, 《解耦的权重衰减正则化》, 载于 ICLR 会议论文集, 2019。
- [24] T. B. Brown 等, 《语言模型是少样本学习器》, 载于 NeurIPS 会议论文集, 2020。
- [25] J. Kaplan 等, 《神经语言模型的缩放定律》, 2020。
- [26] J. Hoffmann 等, 《训练计算最优的大语言模型》, 2022。
- [27] A. Holtzman、J. Buys、L. Du、M. Forbes 和 Y. Choi, 《神经文本退化的奇特现象》, 载于 ICLR 会议论文集, 2020。
- [28] Y.-H. H. Tsai、S. Bai、M. Yamada、L.-P. Morency 和 R. Salakhutdinov, 《Transformer 剖析: 从核方法视角统一理解 Transformer 的注意力》, 载于《2019 年自然语言处理实证方法会议暨第九届国际自然语言处理联合会议论文集》(EMNLP-IJCNLP), K. Inui、J. Jiang、V. Ng 和 X. Wan 编, 中国香港: 计算语言学协会, 2019 年 11 月, 第 4344–4353 页。DOI: 10.18653/v1/D19-1443。
- [29] A. Kazemnejad、I. Padhi、K. Natesan、P. Das 和 S. Reddy, 《位置编码对 Transformer 长度泛化的影响》, 载于第 37 届神经信息处理系统会议, 2023。[在线] 可访问: <https://openreview.net/forum?id=Drrl2gcjz1>